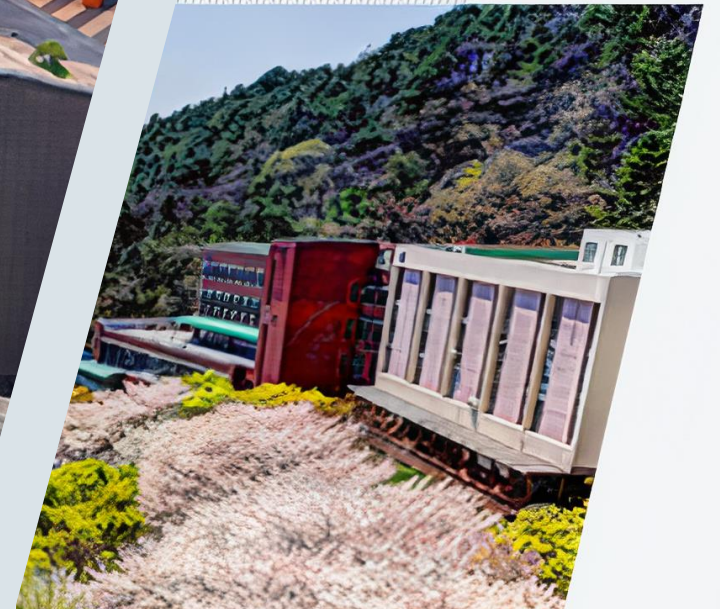


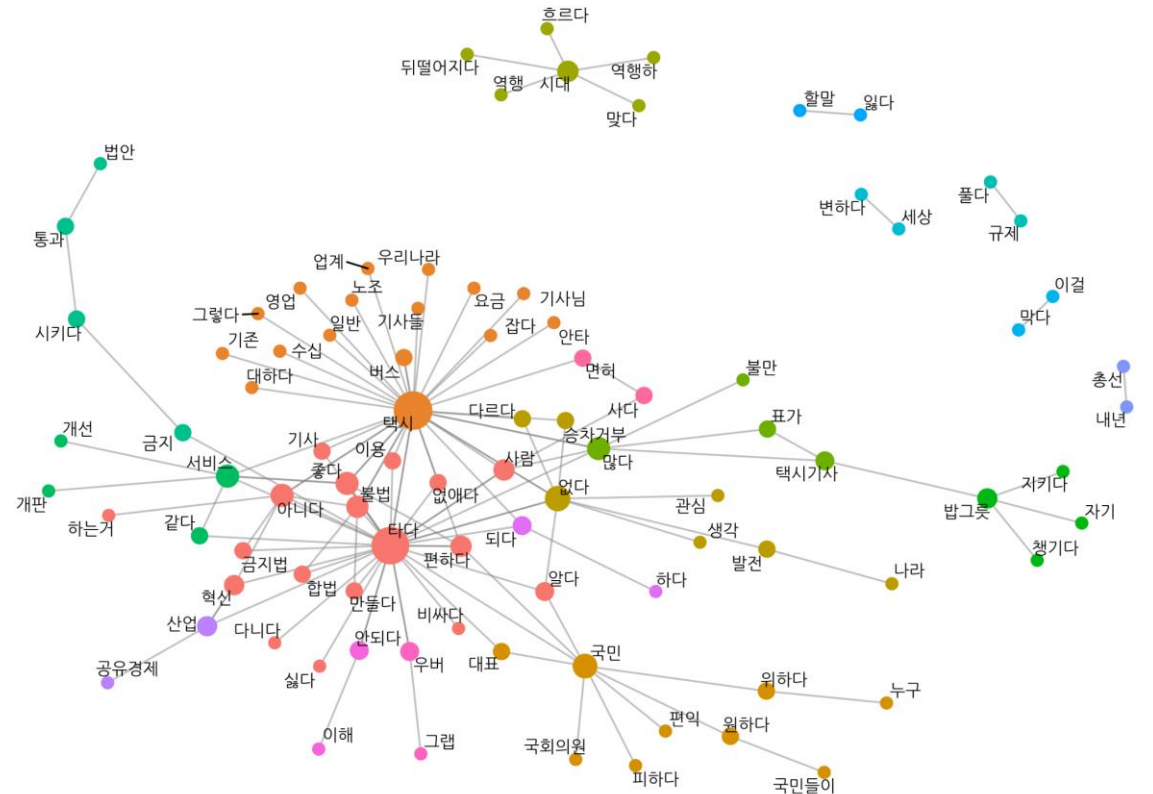
Transformer

컴퓨터AI공학부
2025년 1학기 머신러닝



Background of Language Model

- 정의: 언어라는 현상을 모델링하고자 **단어 시퀀스(문장)에 확률을 할당(assign)**하는 모델
- 목적: 가장 자연스러운 단어 시퀀스를 찾아내는 것
- 방법: **통계(확률)를 이용한 방법**과 **인공 신경망을 이용한 방법**으로 구분



Background of Language Model

- 기본 동작방식: 이전 단어들이 주어졌을 때 다음 단어를 예측

예시 1:

나는 학교에

?



갔다

간다

도착했다

예시 2:

인공지능은 데이터를

?



분석한다

학습한다

좋아한다

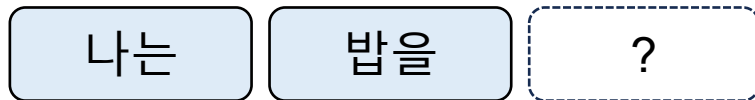
Background of Language Model

- 언어 모델의 유형

1. 자기 회귀 언어 모델

- 이전 단어로부터 다음 단어 예측

예시

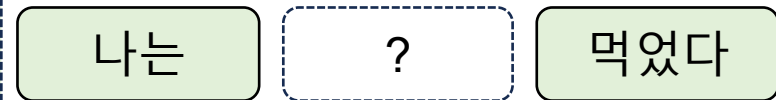


예측 방향

2. 양방향 언어 모델

- 양쪽 문맥을 활용해 가운데 단어 예측
- 최근 딥러닝 기반 모델에서 사용

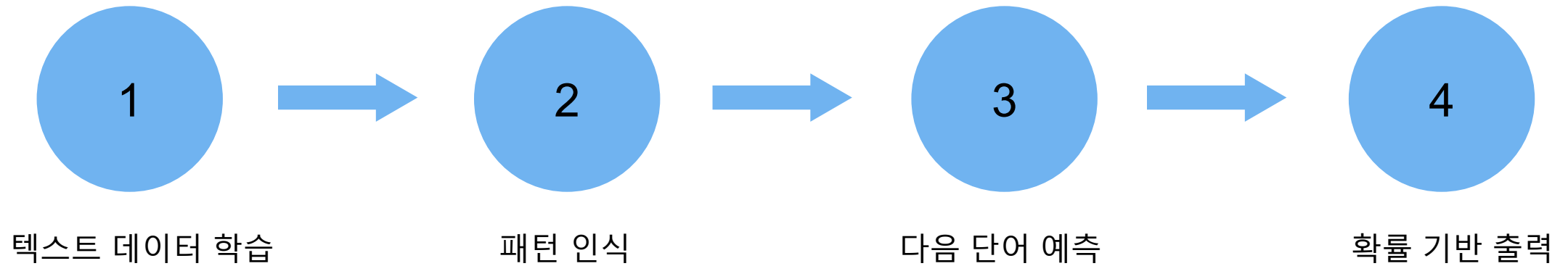
예시



예측 방향

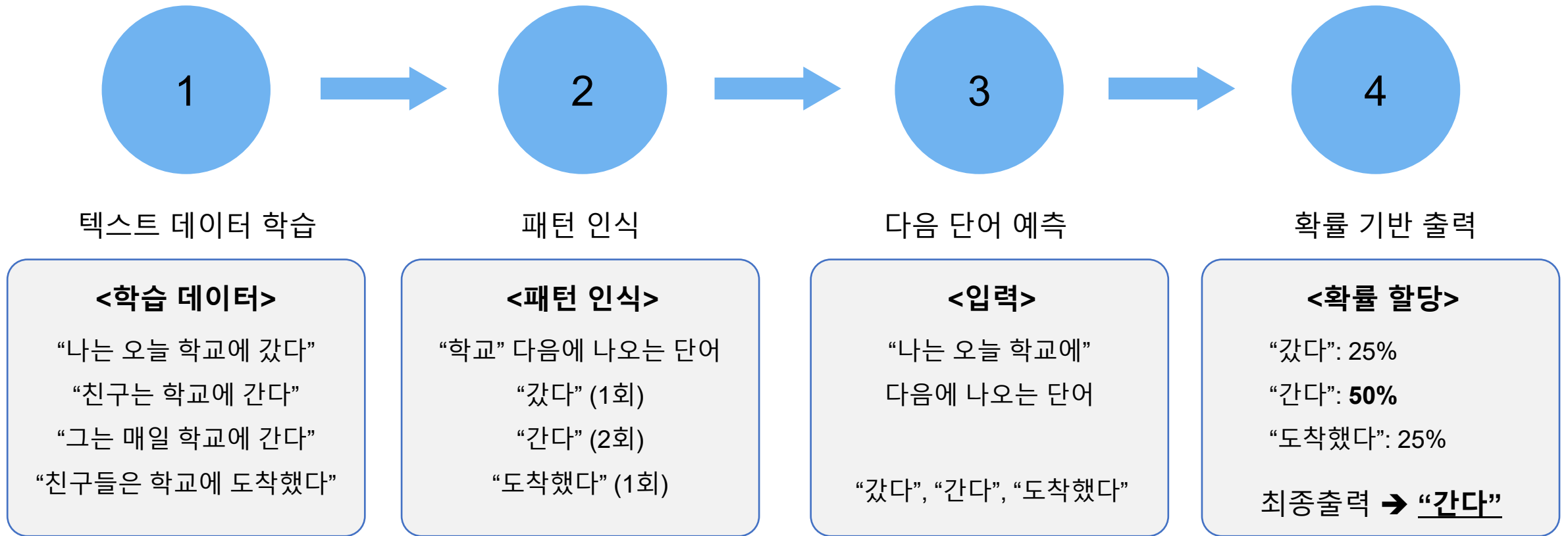
Background of Language Model

- 언어 모델링: 주어진 단어로부터 모르는 단어를 예측하는 작업



Background of Language Model

- 언어 모델링: 주어진 단어로부터 모르는 단어를 예측하는 작업



Background of Language Model

- 언어 모델의 응용

1. 텍스트 자동완성

안녕하세요, 오늘 날씨가 정말 □

2. 챗봇 및 대화 시스템

오늘 날씨 어때요?

오늘은 맑고 따뜻합니다!

3. 콘텐츠 생성

옛날 옛적에...
한 작은 마을에 살던 소년은...
모험을 떠나기로 결심했습니다...

4. 기계 번역

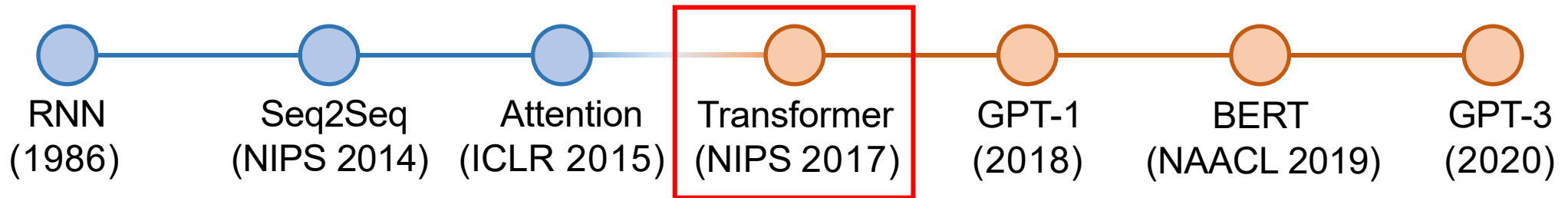
Hello world

안녕 세상

Background of Language Model

- History of Language Model

- Transformer: 최신 고성능 모델들은 해당 아키텍처 기반으로 설계



*BERT: Bidirectional Encoder Representations from Transformers

*GPT: Generative Pre-trained Transformer

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

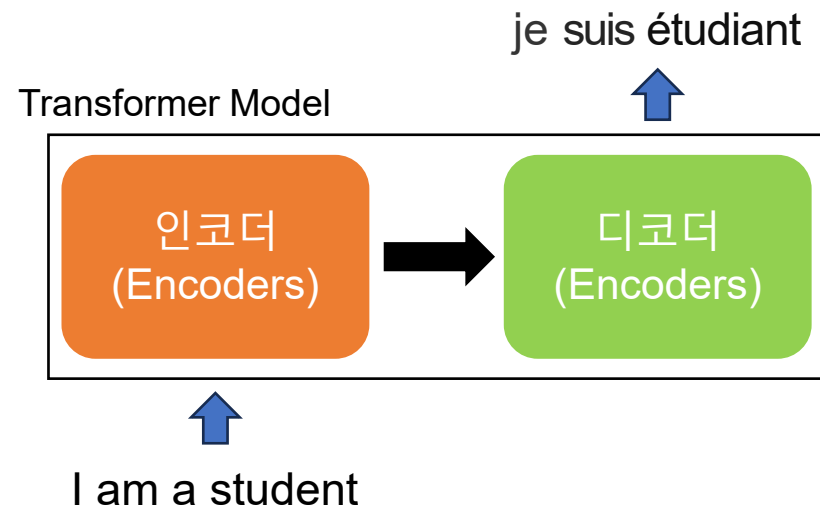
Aidan N. Gomez*[†]
University of Toronto
aidan@cs.toronto.edu

Łukasz Kaiser*
Google Brain
lukaszkaizer@google.com

Illia Polosukhin*[‡]
illia.polosukhin@gmail.com

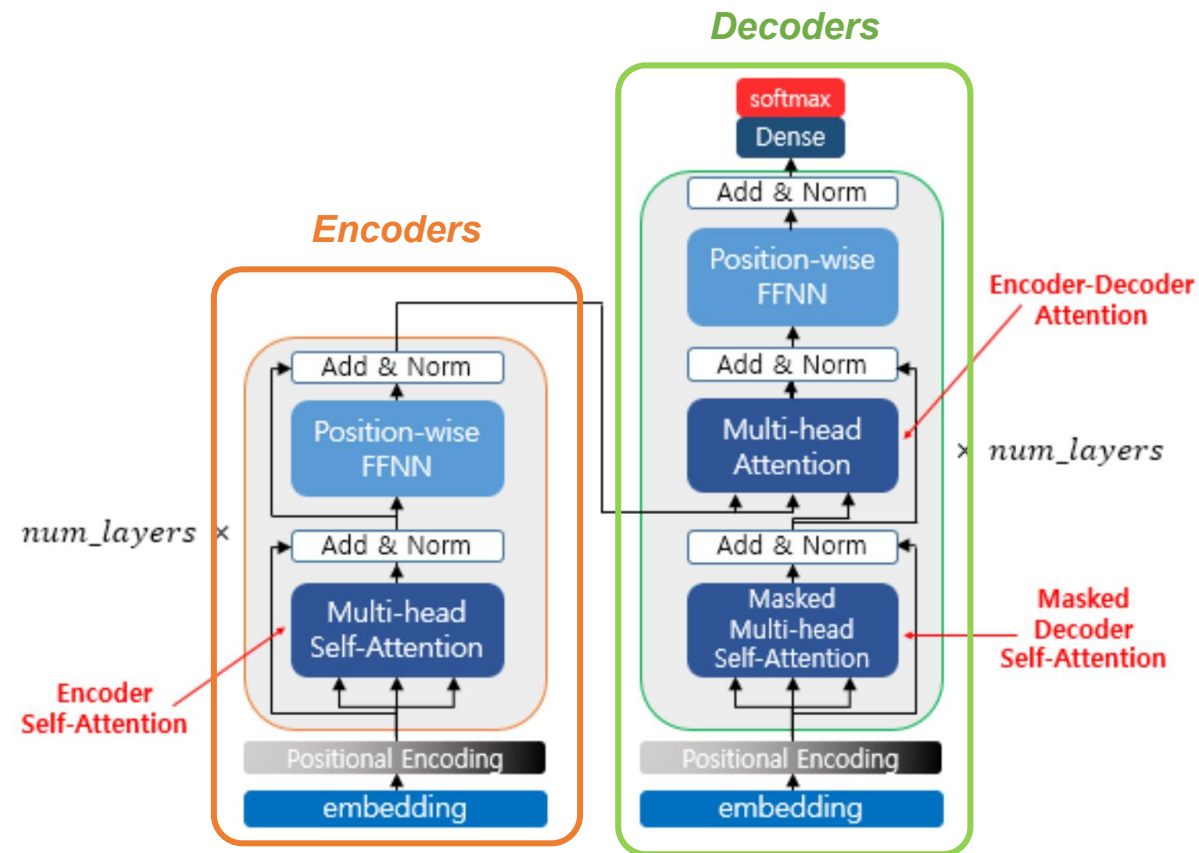
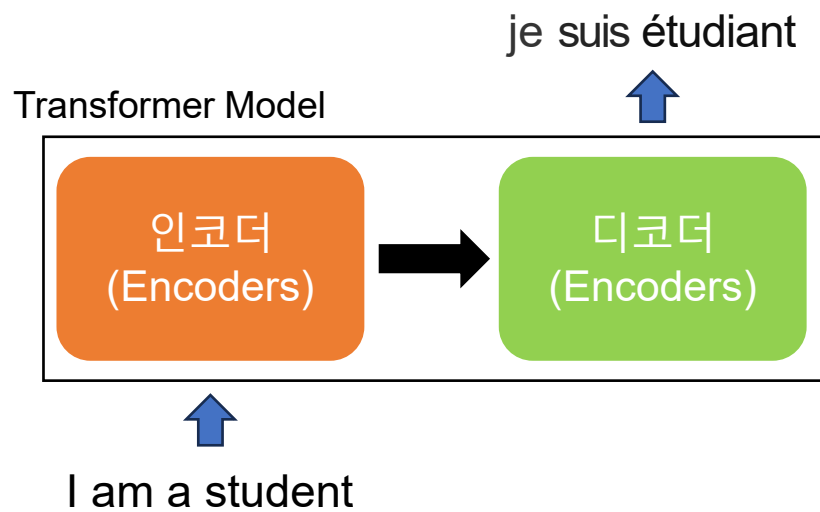
Transformer

- 전체 구조 기계번역 (영어→프랑스어)
 - 입력: 영어 문장
 - 출력: 프랑스어 문장



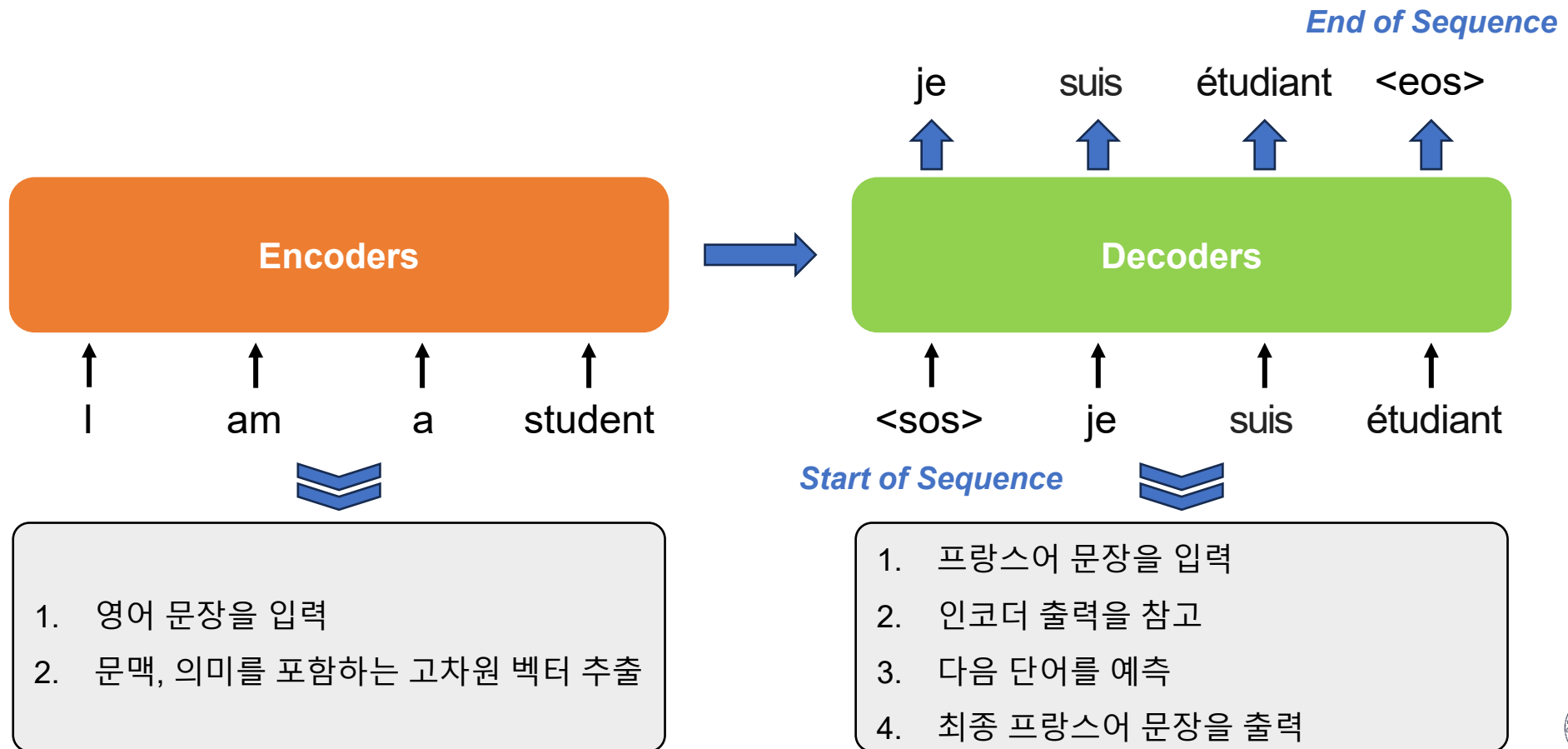
Transformer

- 전체 구조 기계번역 (영어→프랑스어)
 - 입력: 영어 문장
 - 출력: 프랑스어 문장



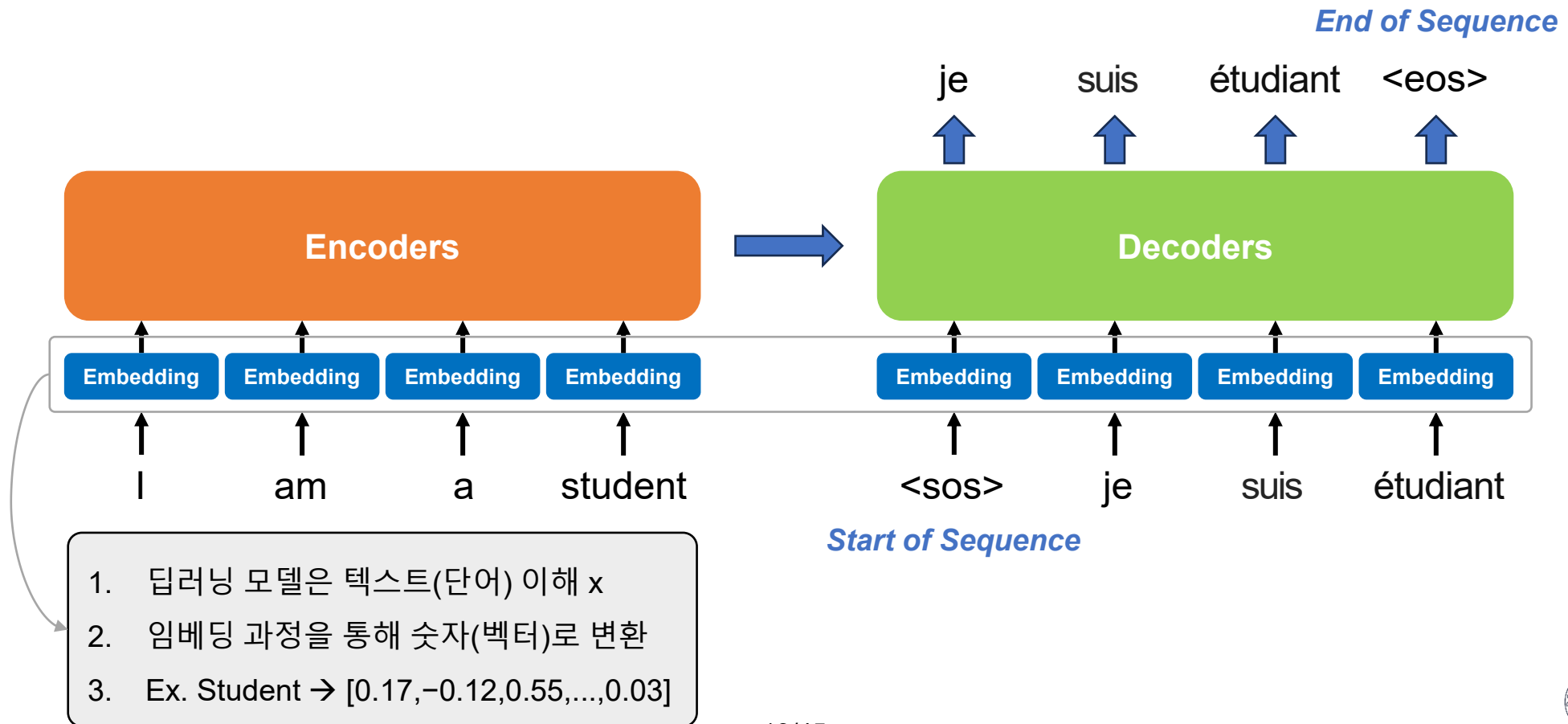
Transformer

- 전체 구조 기계번역 (영어→프랑스어)
 - 학습 시 입력: 영어, 프랑스어 문장
 - 학습 시 출력: 프랑스어 문장



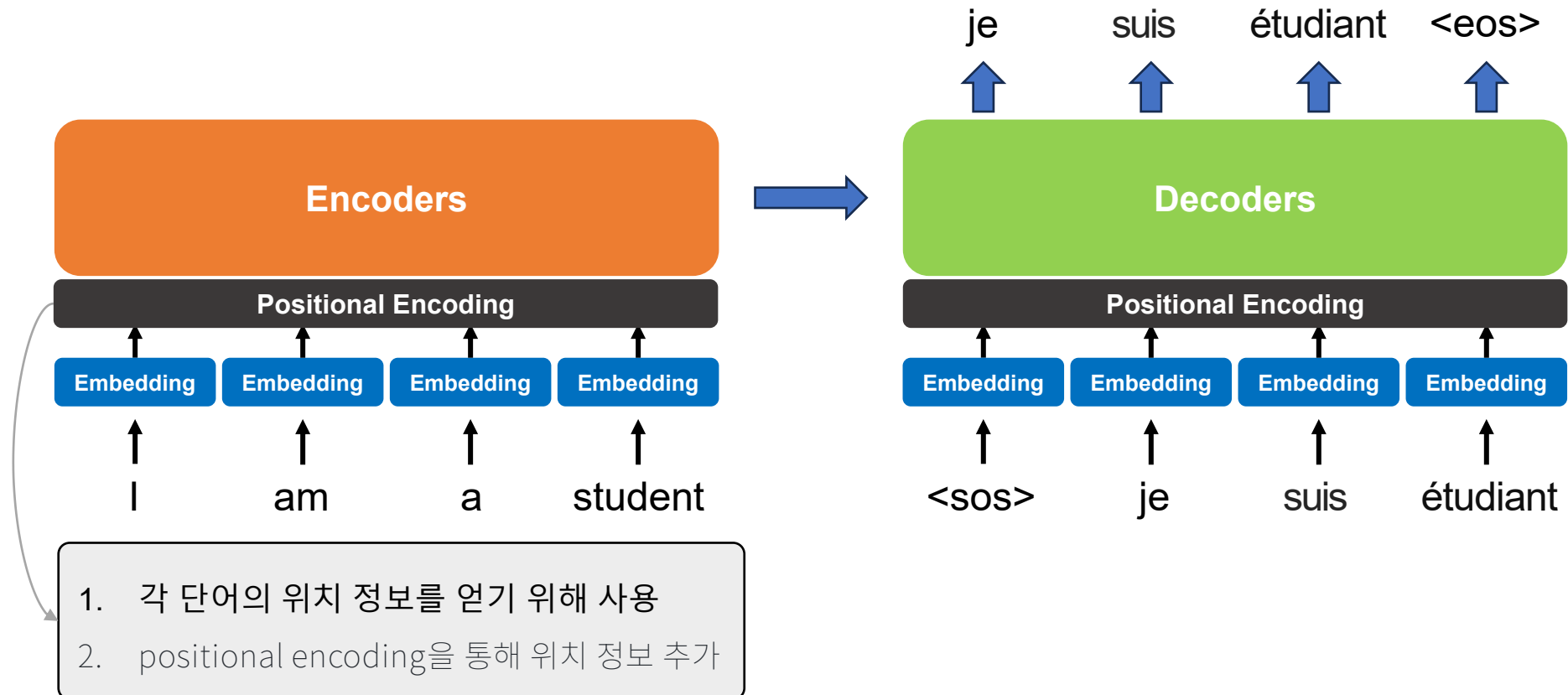
Transformer

- Embedding



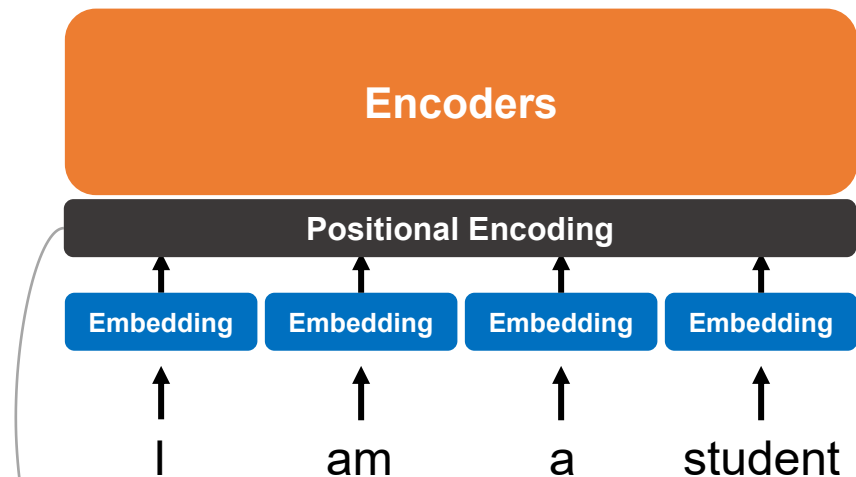
Transformer

- Positional Encoding



Transformer

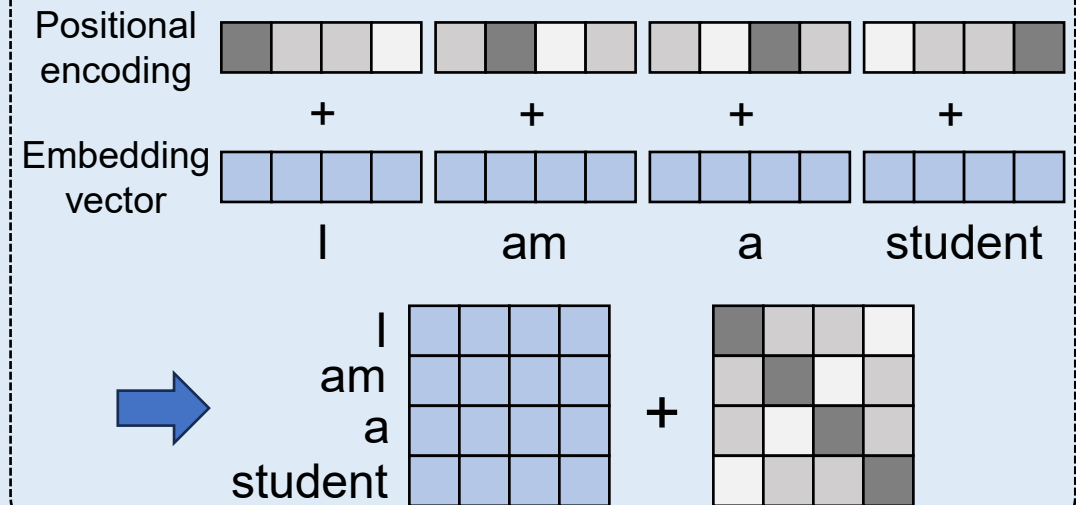
Positional Encoding



1. 각 단어의 위치 정보를 얻기 위해 사용
2. positional encoding을 통해 위치 정보 추가

Ex. "I love you" vs "You love I"

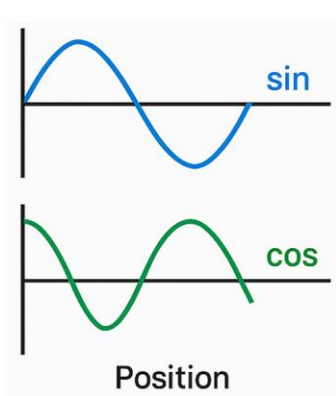
- 단어는 동일하지만, 순서에 따라 의미가 완전히 달라짐
- ["I", "love", "you"] → 3개의 벡터가 동시에 처리 → 위치x
- 순서 정보가 없으면 트랜스포머는 두 문장을 같다고 인식



- 위치 정보 추가를 위해 덧셈 연산 진행
- 임베딩 벡터 위치, 차원 인식 가능

Transformer

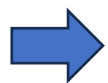
Positional Encoding



$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{model}})$$

$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{model}})$$

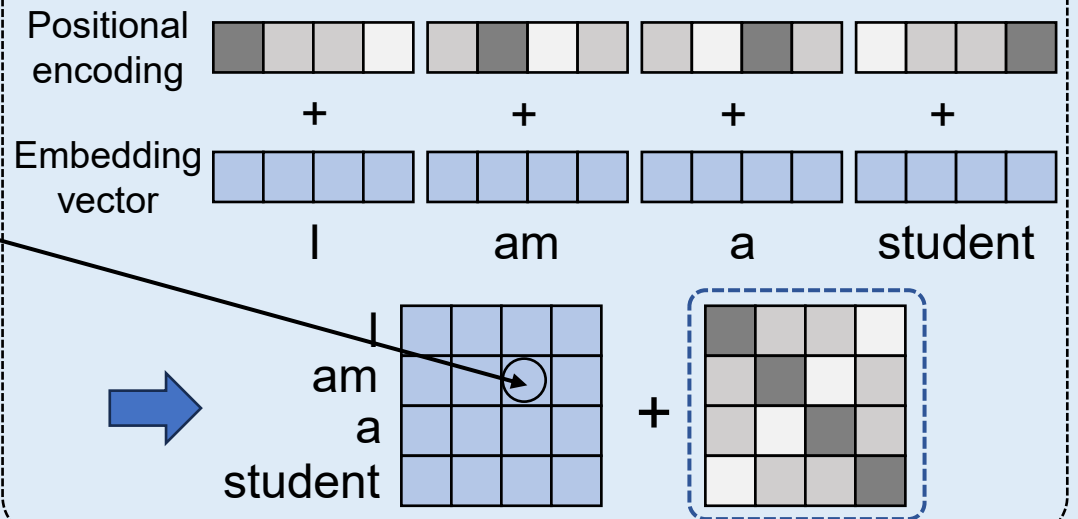
- ✓ pos : 위치 인덱스 (ex. 0, 1, 2, ...)
- ✓ i : 벡터 차원 인덱스 (ex. 0부터 시작)
- ✓ d_{model} : 전체 임베딩 차원 수 (ex. 4)



Embedding vector의 정보 기반으로
Positional encoding 정보 생성

Ex. "I love you" vs "You love I"

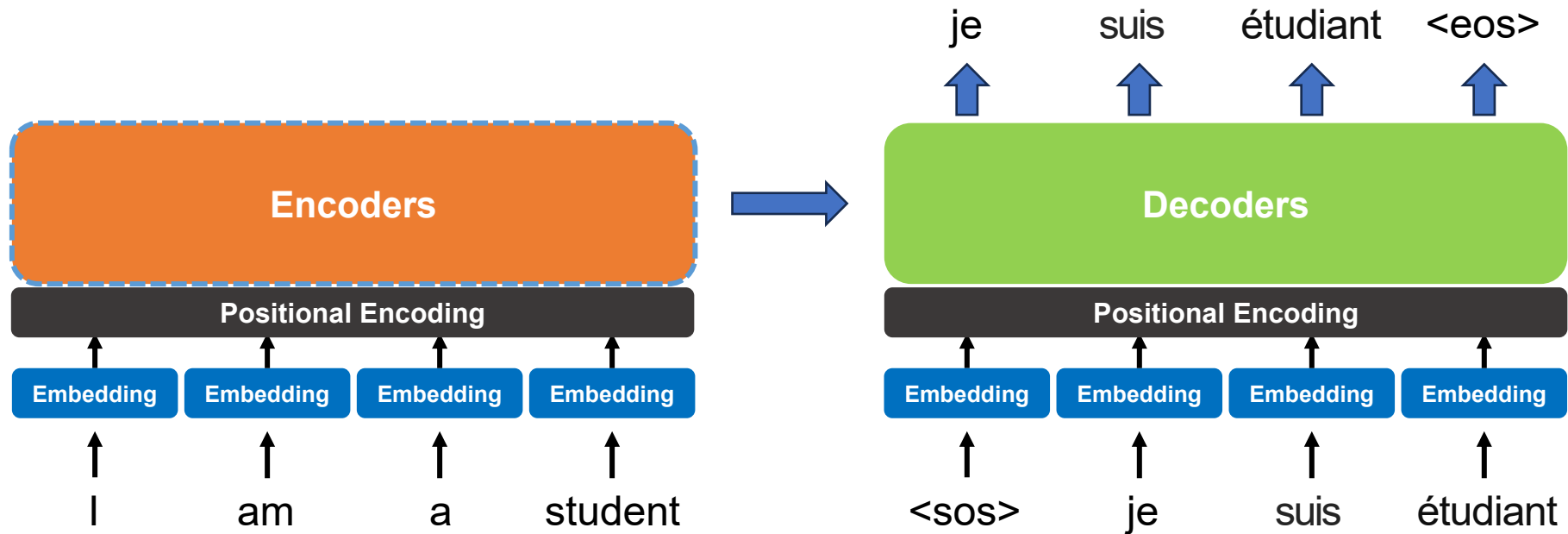
- 단어는 동일하지만, 순서에 따라 의미가 완전히 달라짐
- ["I", "love", "you"] → 3개의 벡터가 동시에 처리 → 위치x
- 순서 정보가 없으면 트랜스포머는 두 문장을 같다고 인식



- 위치 정보 추가를 위해 덧셈 연산 진행
- 임베딩 벡터 위치, 차원 인식 가능

Transformer

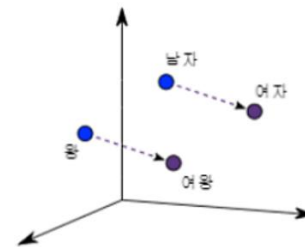
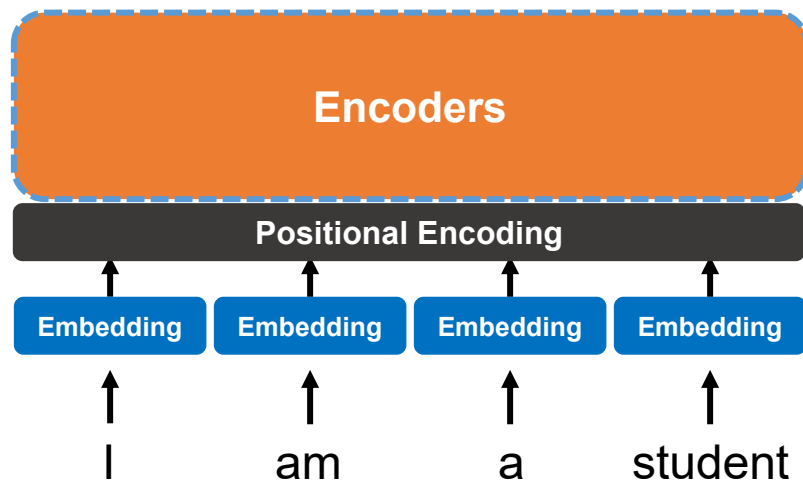
- Encoders



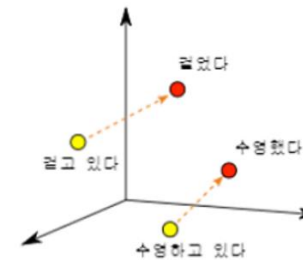
Transformer

Encoders

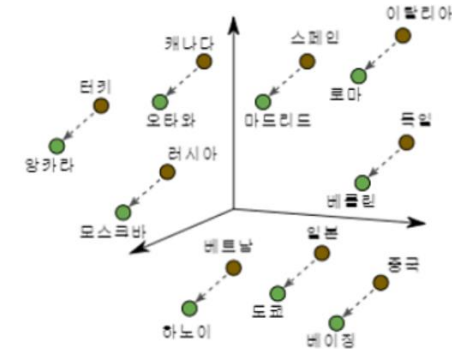
- 자연어를 벡터로 변환한 뒤 단어간 유사도 (Relevance) 표현



남자-여자



동사 시제

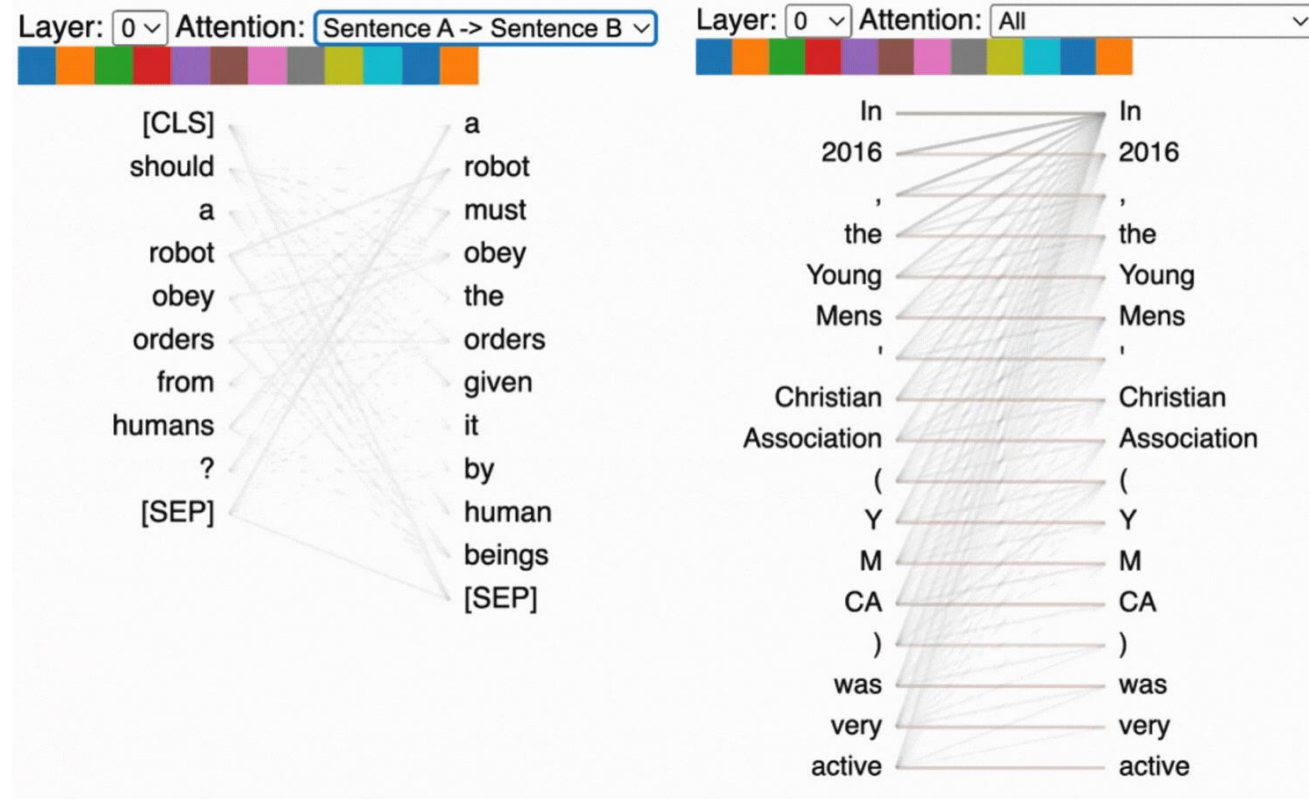
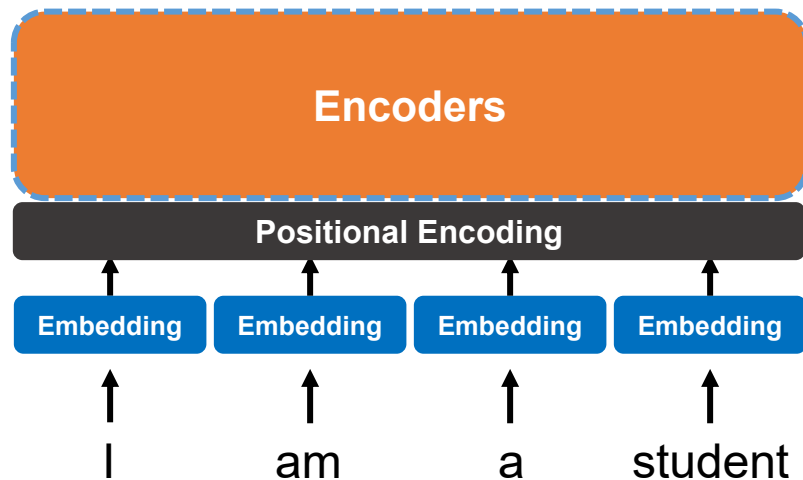


국가-수도

Transformer

Encoders

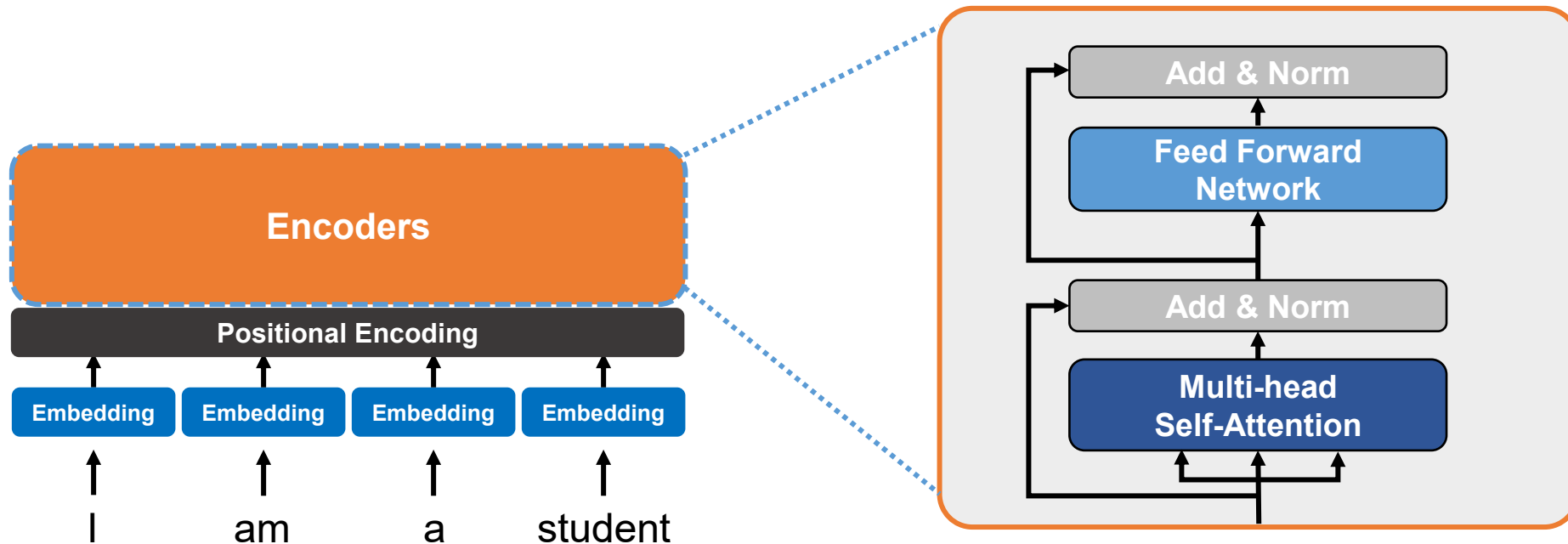
- 자연어를 벡터로 변환한 뒤 단어간 유사도 (Relevance) 표현
- 각각의 단어별 연관성이 높은 정도를 학습 → 단어별 Attention을 통해 문맥에 대한 정보 학습



Transformer

▪ Encoders

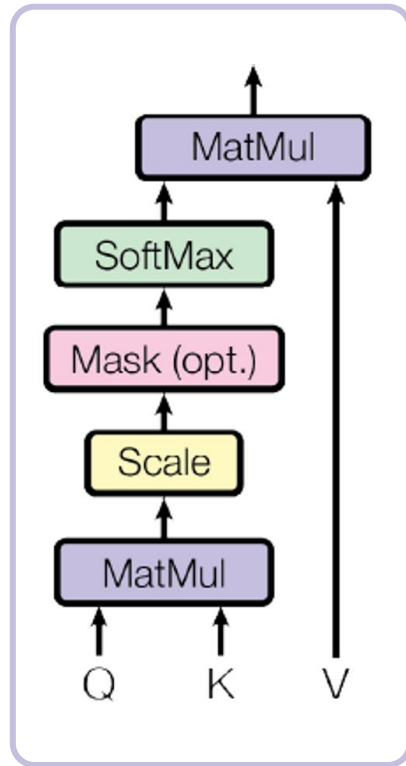
- 자연어를 벡터로 변환한 뒤 단어간 유사도 (Relevance) 표현
- 각각의 단어별 연관성이 높은 정도를 학습 → 단어별 Attention을 통해 문맥에 대한 정보 학습



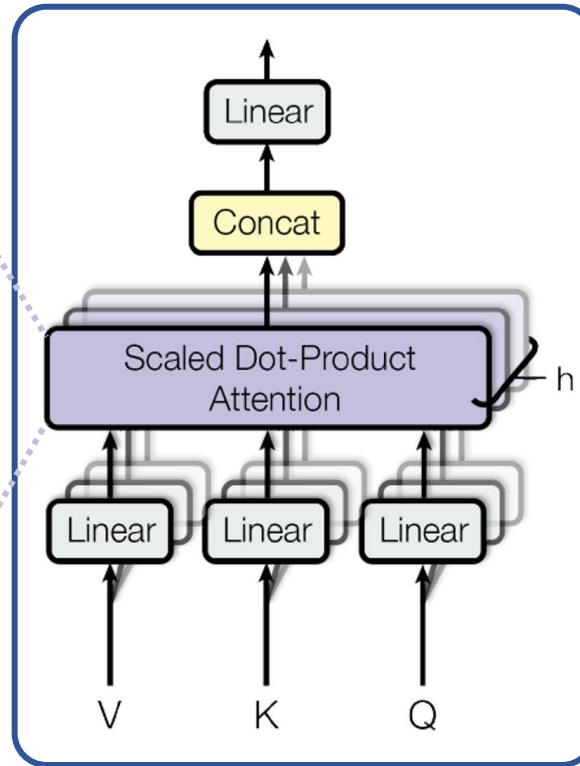
Transformer

Encoders

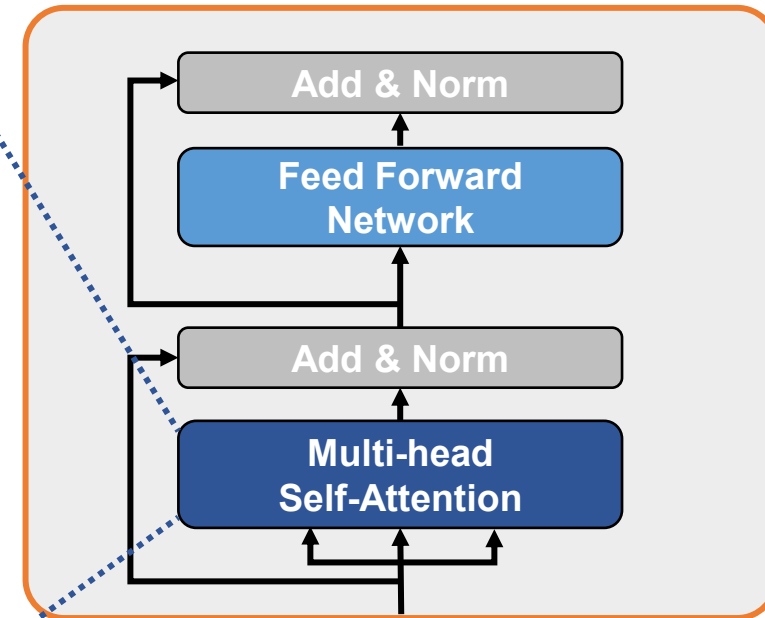
- Multi-head Self-Attention
- Scaled Dot-Product Attention



Scaled Dot-Product Attention



Multi-head Self-Attention



- ✓ Query (Q): 비교 기준
- ✓ Key (K): 비교 대상
- ✓ Value (V): 참조 정보

Transformer

▪ Encoder

• Scaled Dot-Product Attention

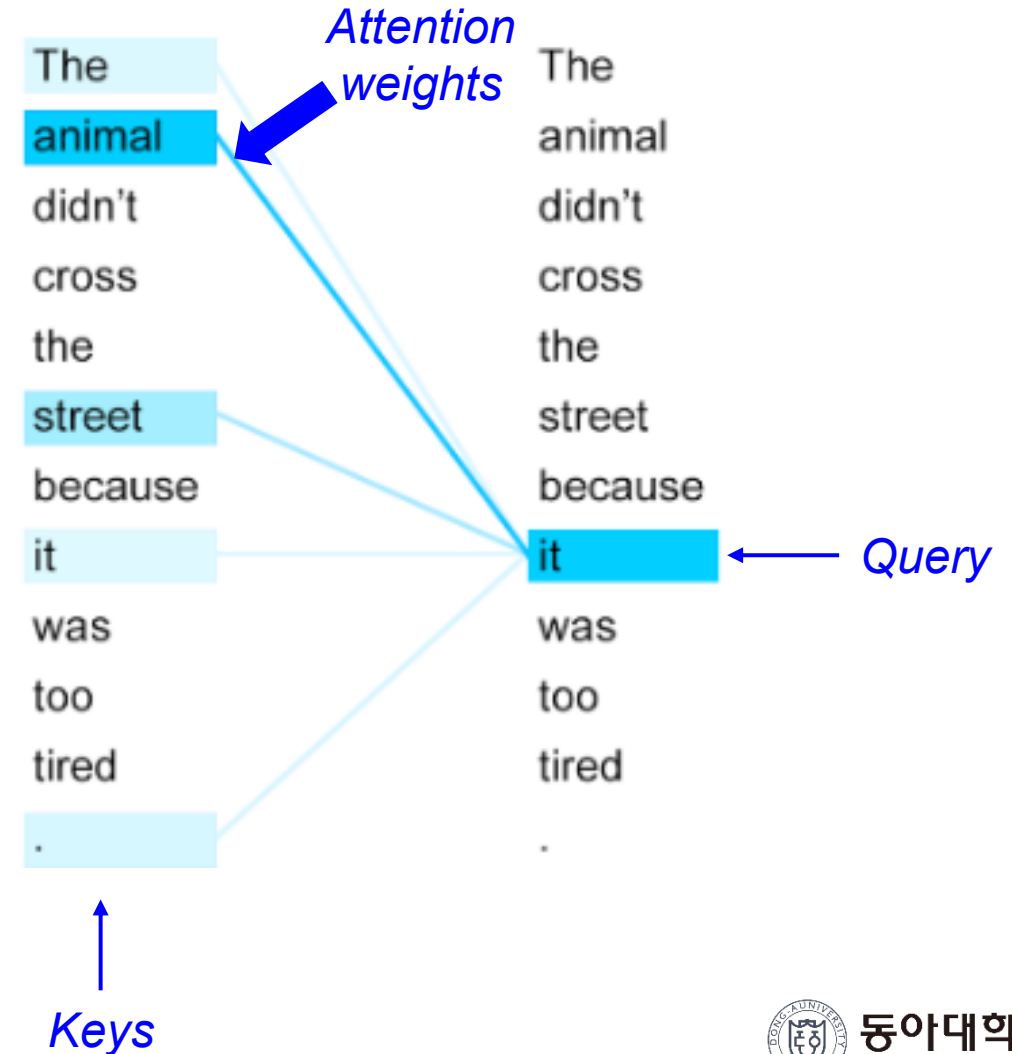
- ✓ Q : Query → 분석 대상이 되는 단어에 대한 가중치 Vector
- ✓ K : Key → Attention을 수행할 단어 집합
- ✓ V : Value → Attention으로 도출된 weights를 적용한 단어

❖ 예시) 영어 → 한국어 (Self-attention의 목적 기준)

The animal didn't cross the street because it was too tired.
그 동물은 길을 건너지 않았다.

기계의 경우 “it”이 “street”인지 “animal”인지 모름

→ 입력 문장내의 단어들끼리 유사도를 계산하여 가중치 부여

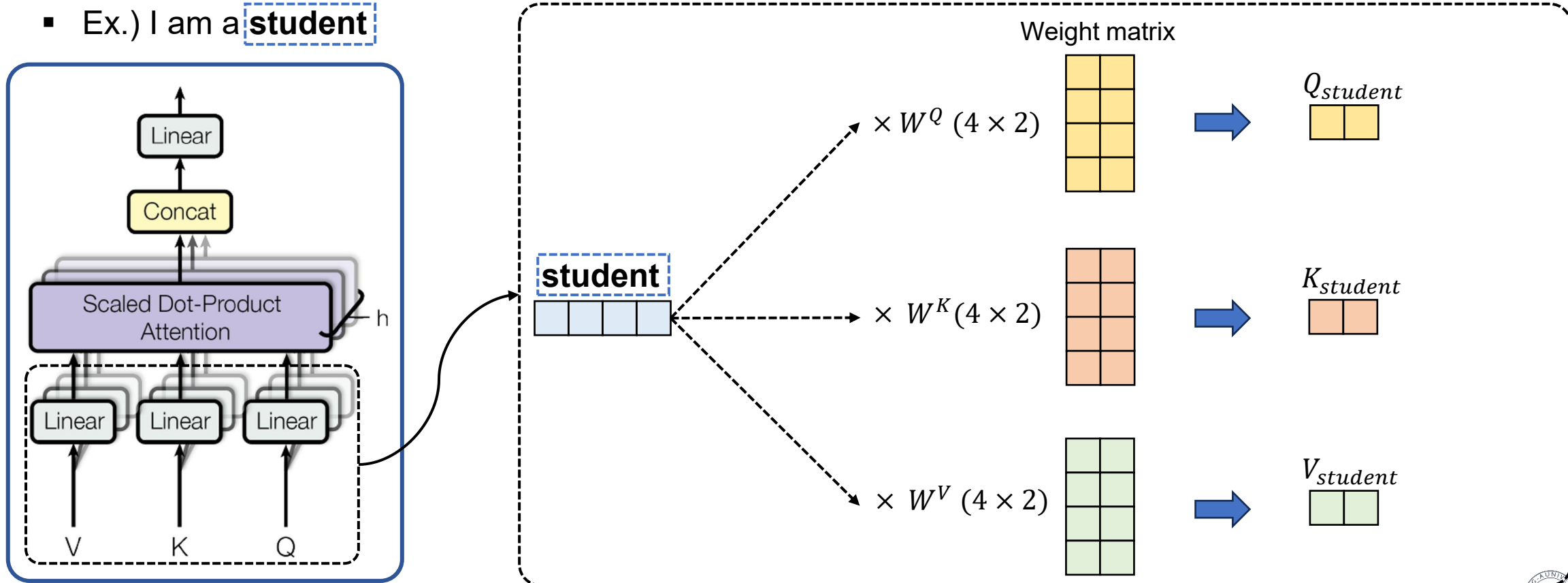


Transformer

- Encoders

- Multi-head Self-Attention
- Scaled Dot-Product Attention

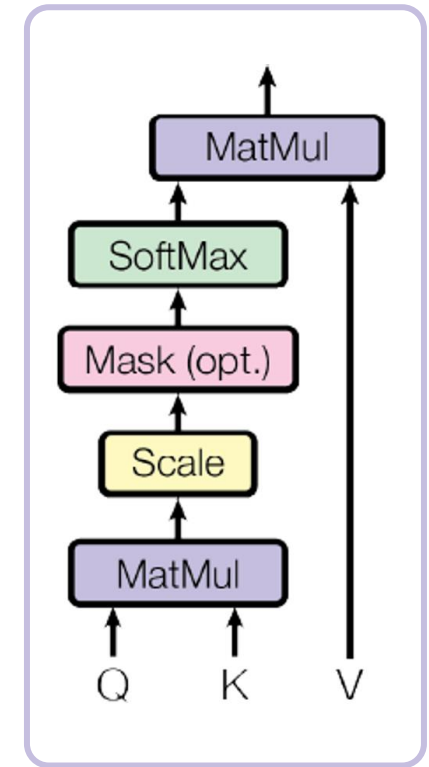
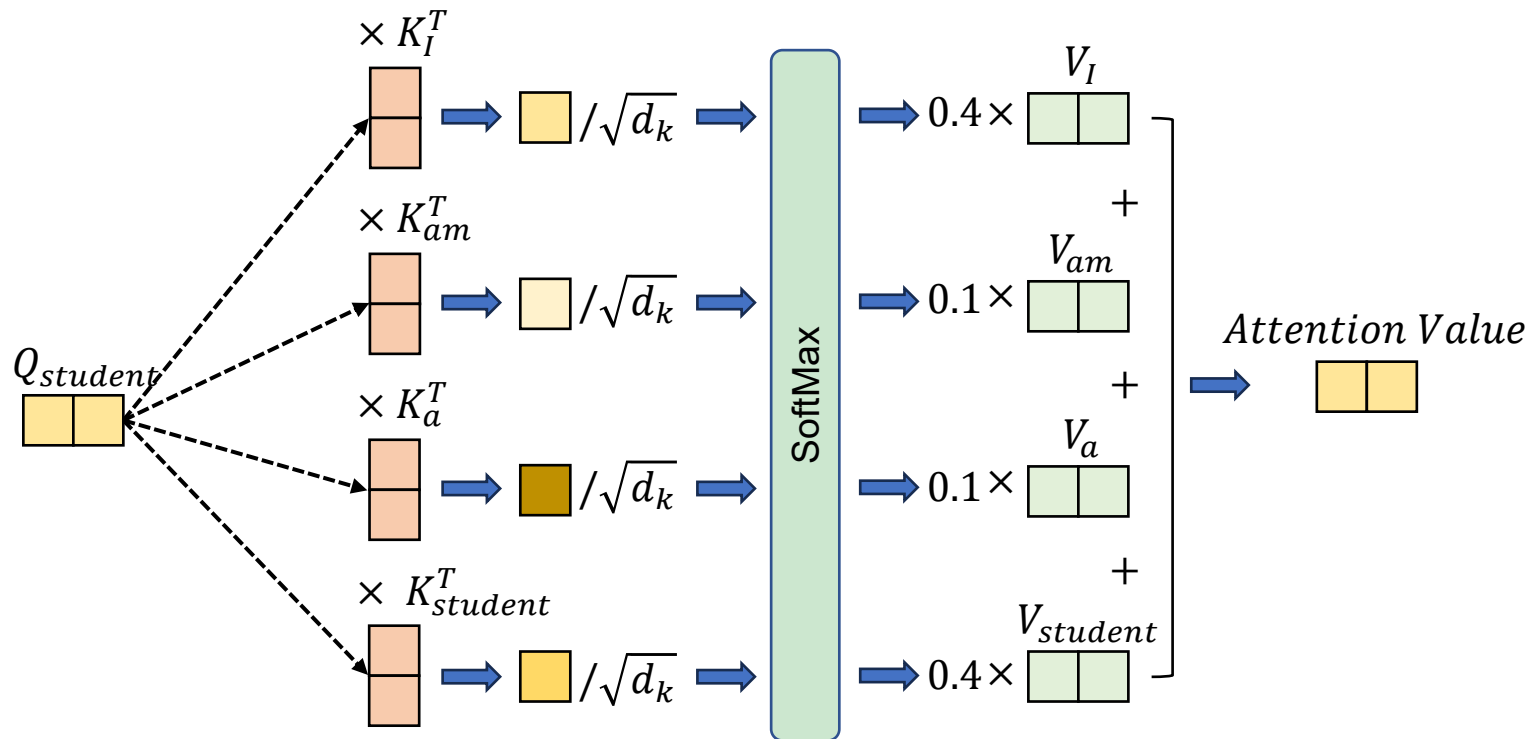
- Ex.) I am a **student**



Multi-head Self-Attention

Transformer

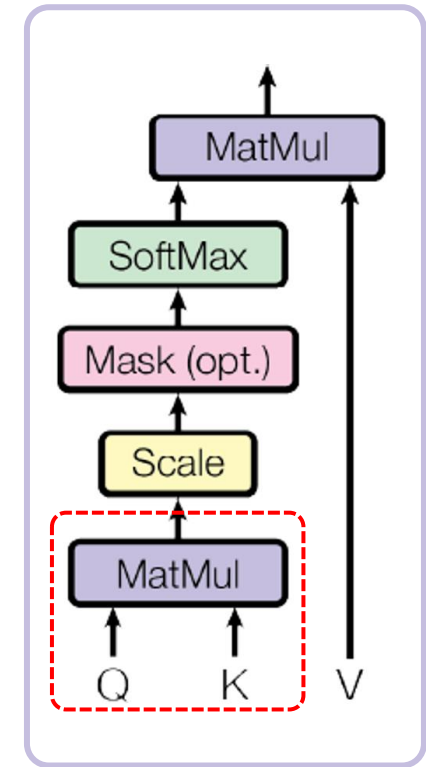
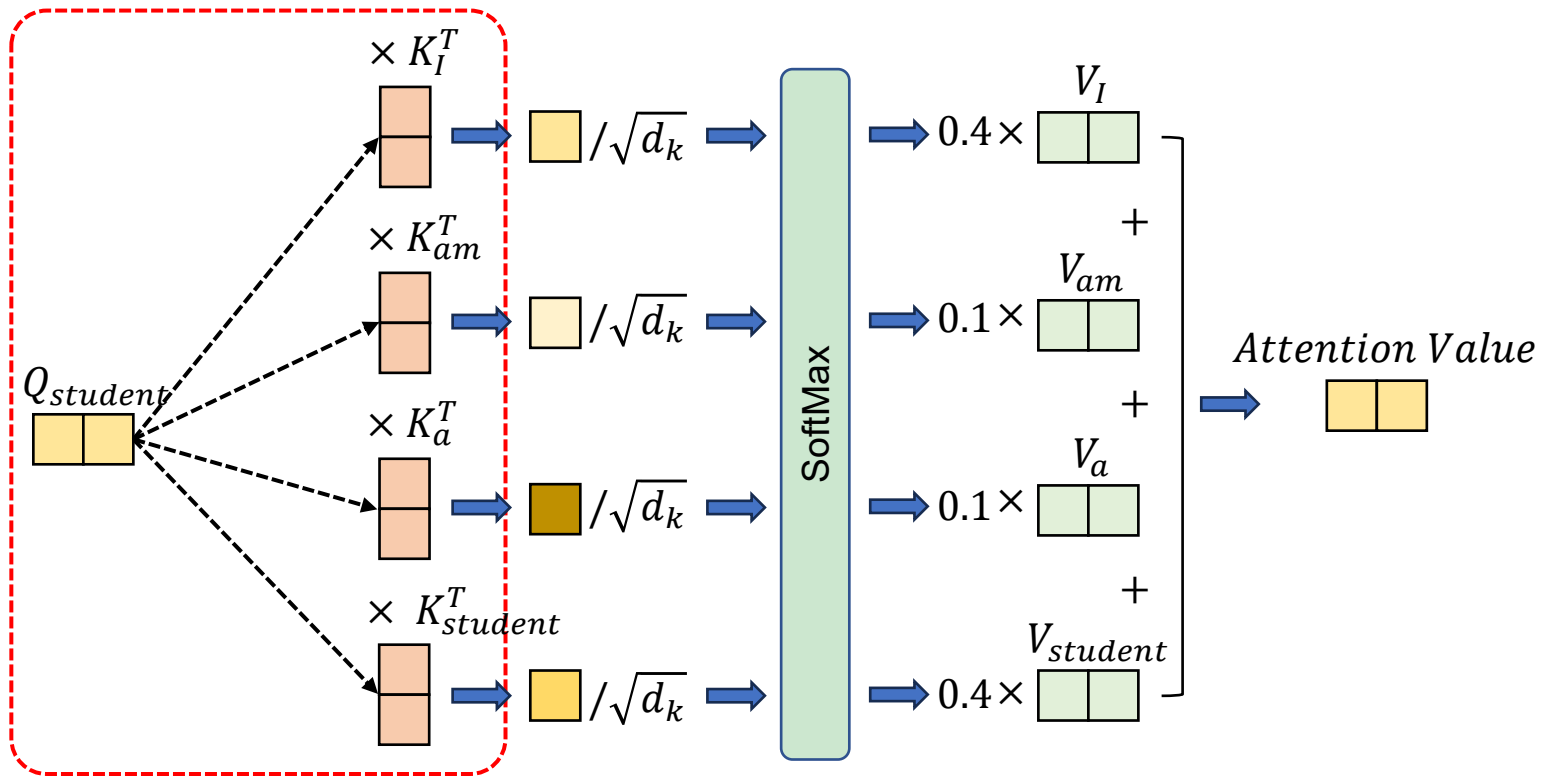
- Encoders
 - Multi-head Self-Attention
 - Scaled Dot-Product Attention
- Ex.) I am a **student**



Scaled Dot-Product Attention

Transformer

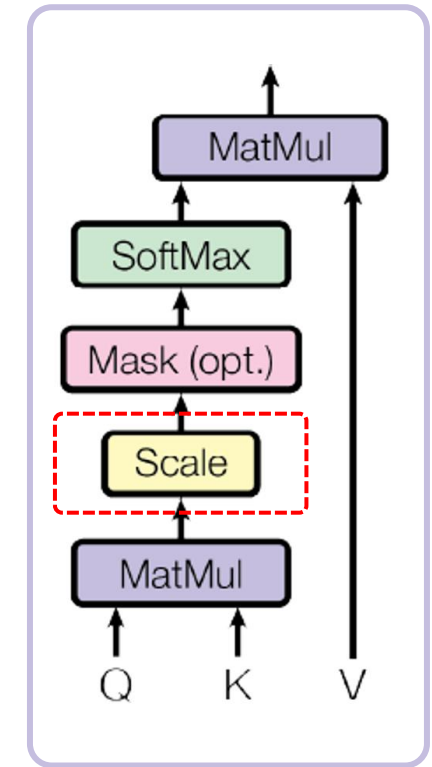
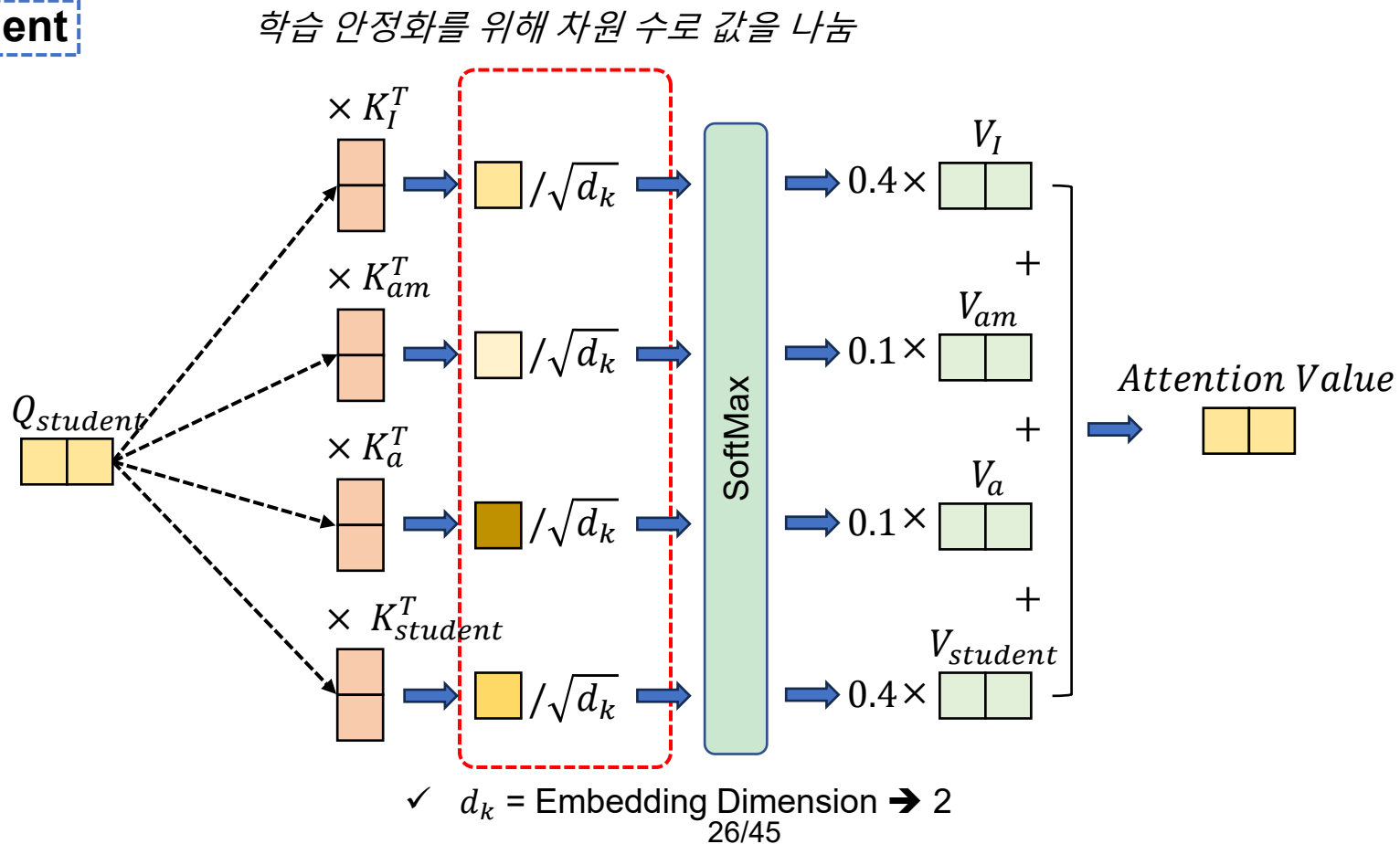
- Encoders
 - Multi-head Self-Attention
 - Scaled Dot-Product Attention
- Ex.) I am a **student**



Scaled Dot-Product Attention

Transformer

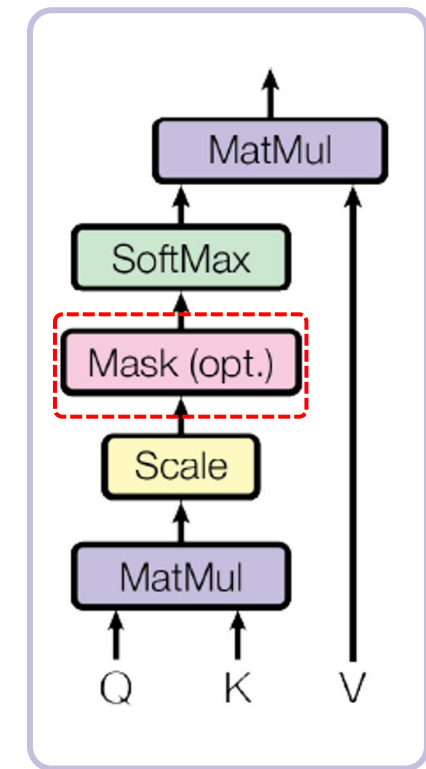
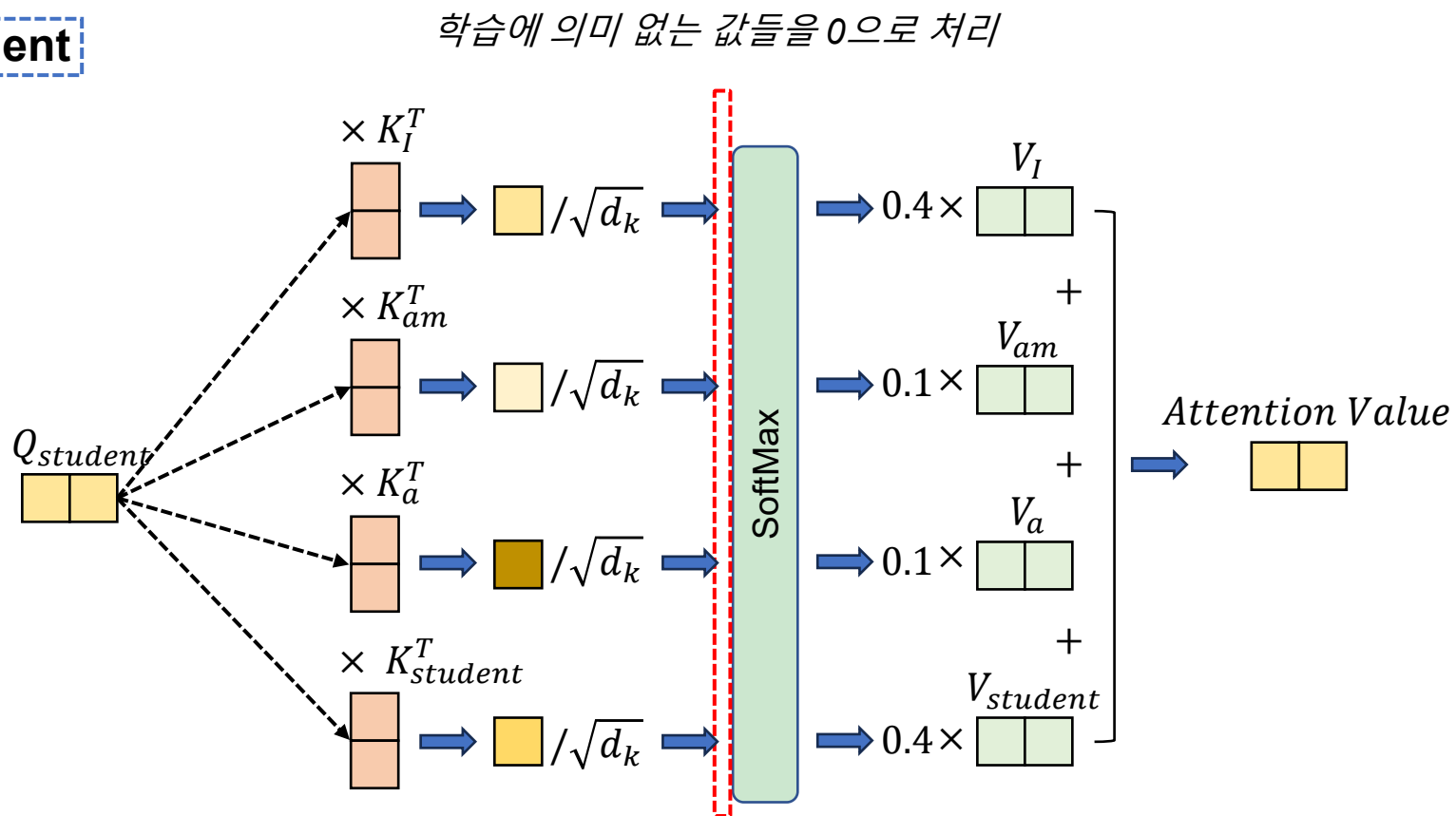
- Encoders
 - Multi-head Self-Attention
 - Scaled Dot-Product Attention
- Ex.) I am a **student**



Scaled Dot-Product Attention

Transformer

- Encoders
 - Multi-head Self-Attention
 - Scaled Dot-Product Attention
- Ex.) I am a **student**



Scaled Dot-Product Attention

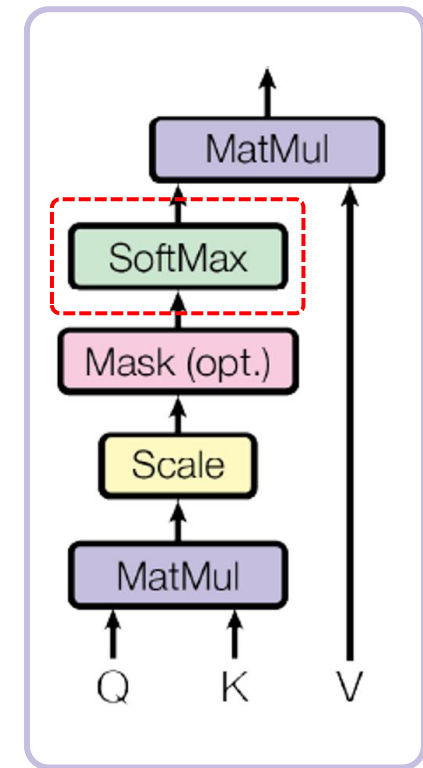
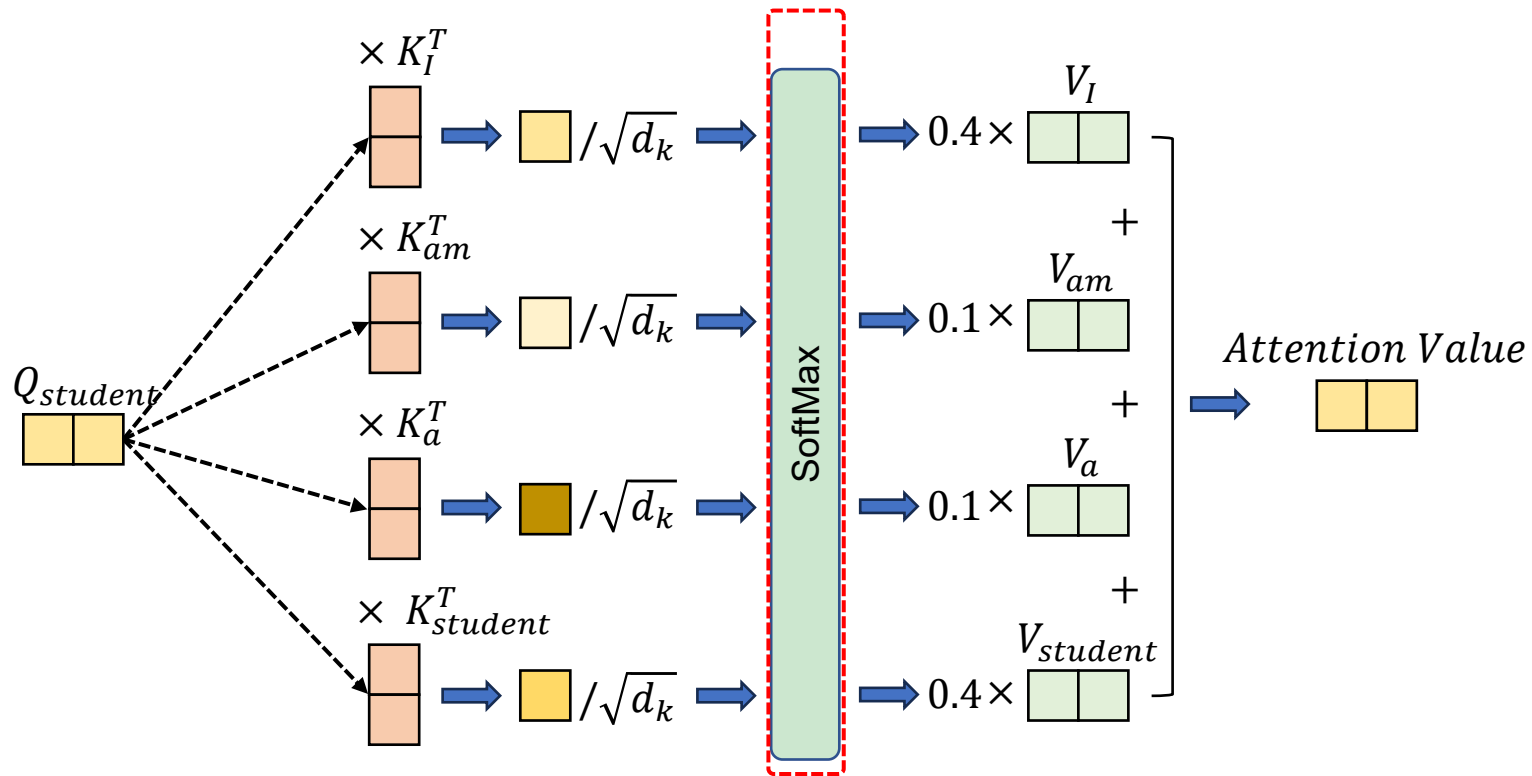
Transformer

- Encoders

- Multi-head Self-Attention
- Scaled Dot-Product Attention

- Ex.) I am a **student**

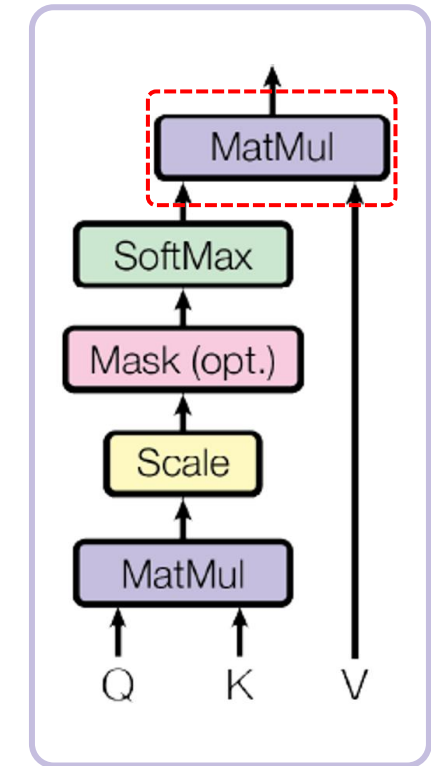
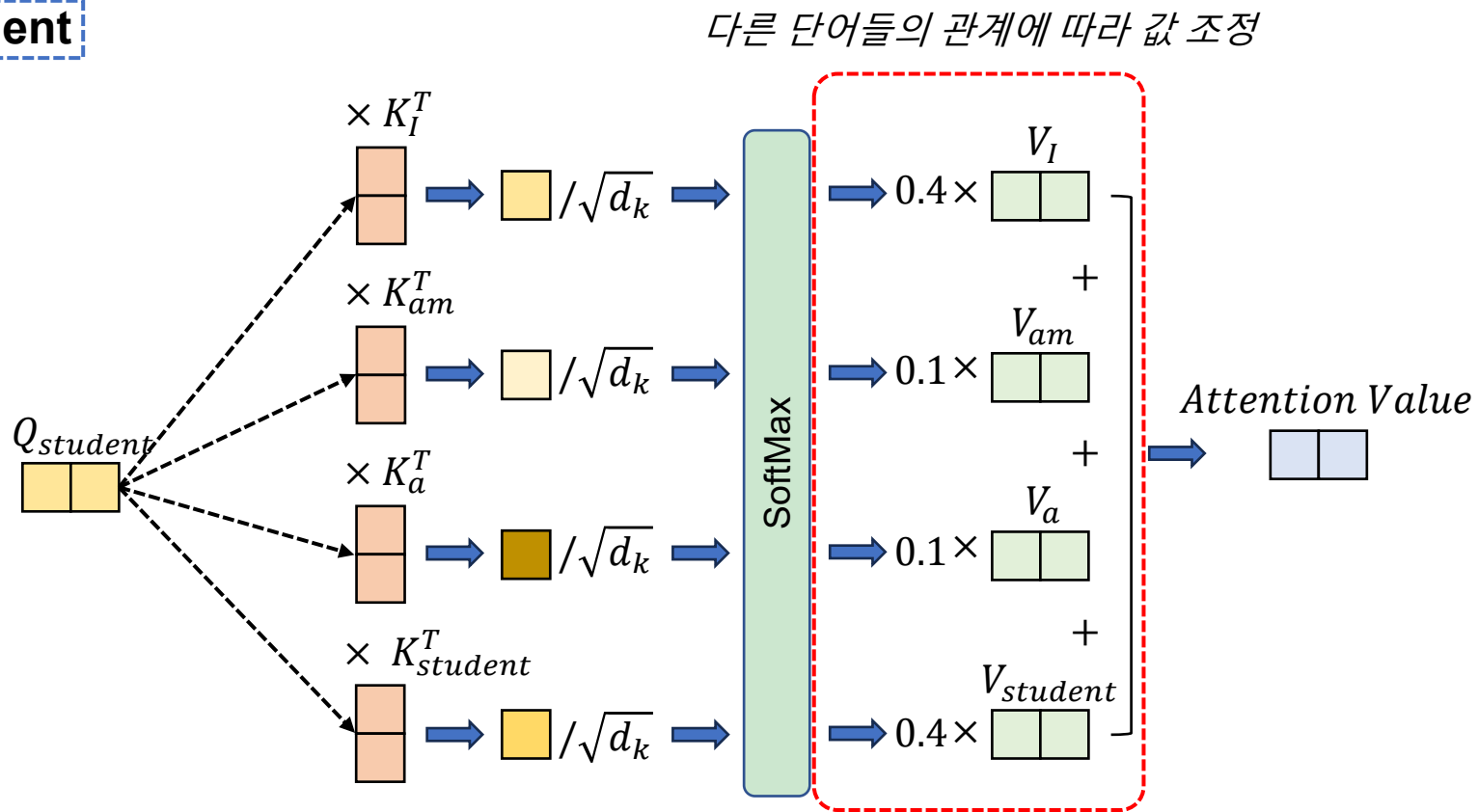
차원 수가 깊어 질수록 미분 값이 작아지는 현상을 방지,
확률 값 출력



Scaled Dot-Product Attention

Transformer

- Encoders
 - Multi-head Self-Attention
 - Scaled Dot-Product Attention
- Ex.) I am a **student**

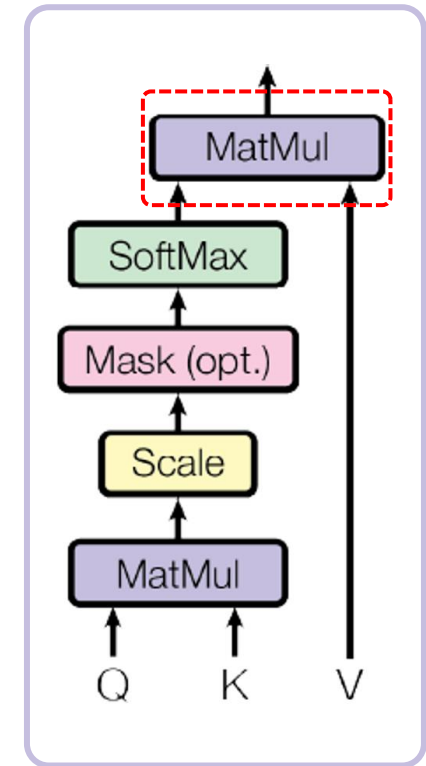


Scaled Dot-Product Attention

Transformer

- Encoders
 - Multi-head Self-Attention
 - Scaled Dot-Product Attention
- Ex.) I am a **student**

$$Attention(Q, K, V) = softmax(\frac{QK^T}{\sqrt{d_k}})V \quad \ll$$



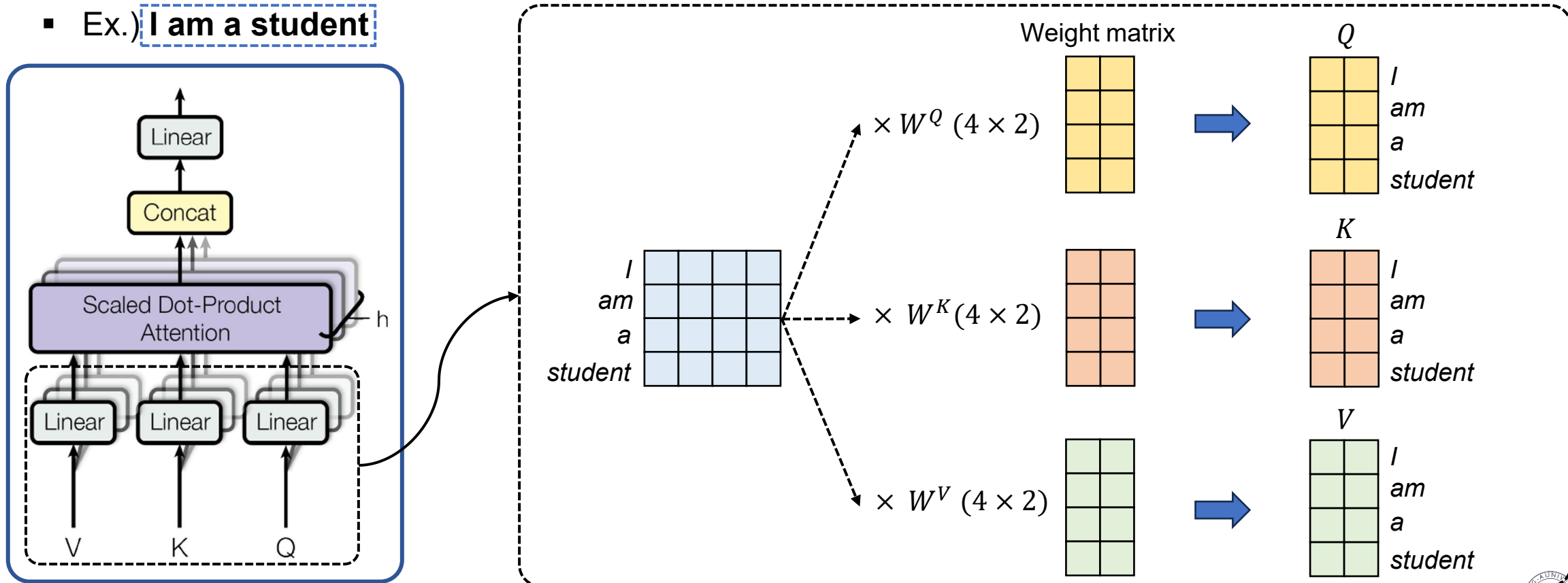
Scaled Dot-Product Attention

Transformer

- Encoders

- Multi-head Self-Attention
- Scaled Dot-Product Attention

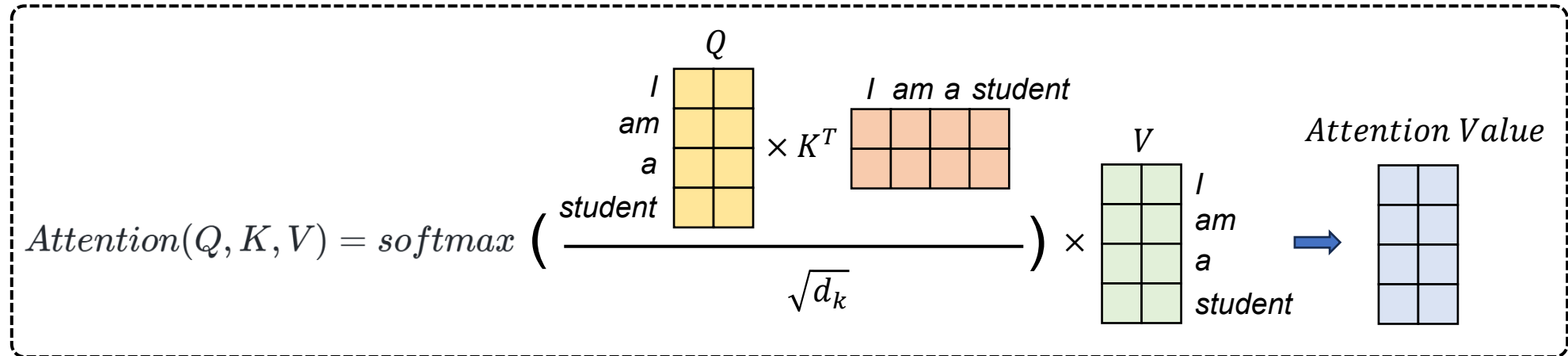
- Ex.) **I am a student**



Multi-head Self-Attention

Transformer

- Encoders
 - Multi-head Self-Attention
 - Scaled Dot-Product Attention
- Ex.) **I am a student**

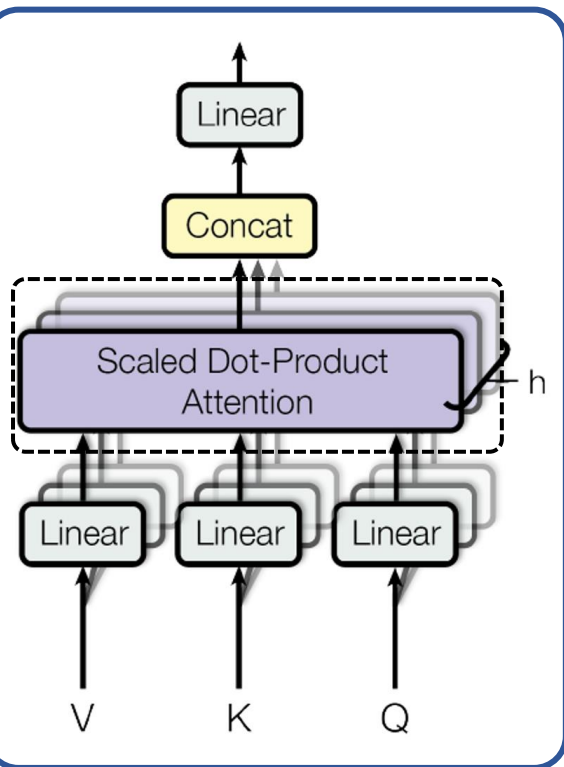


Transformer

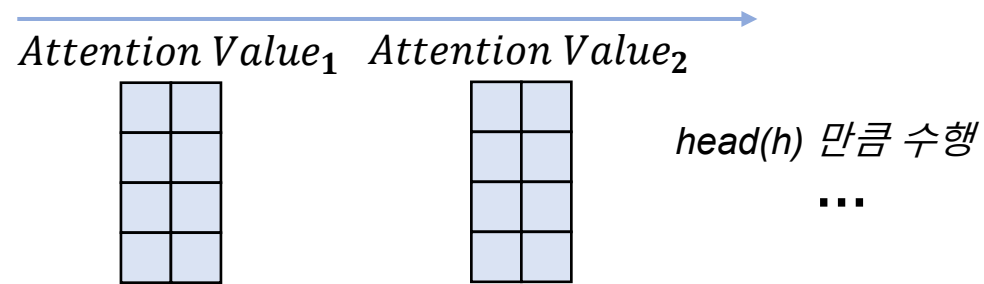
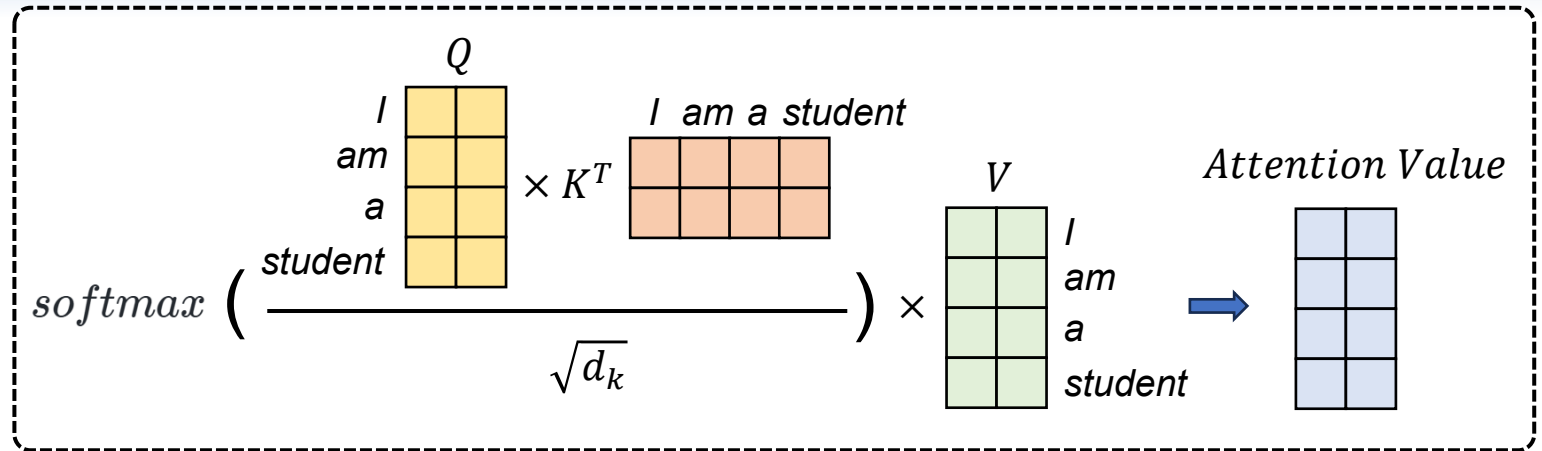
Encoders

- Multi-head Self-Attention
- Scaled Dot-Product Attention

Ex.) I am a student



Multi-head Self-Attention

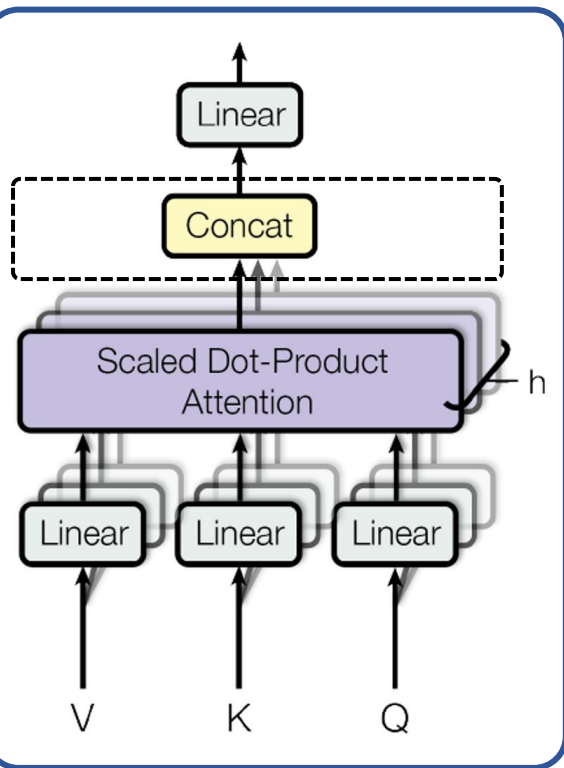


Transformer

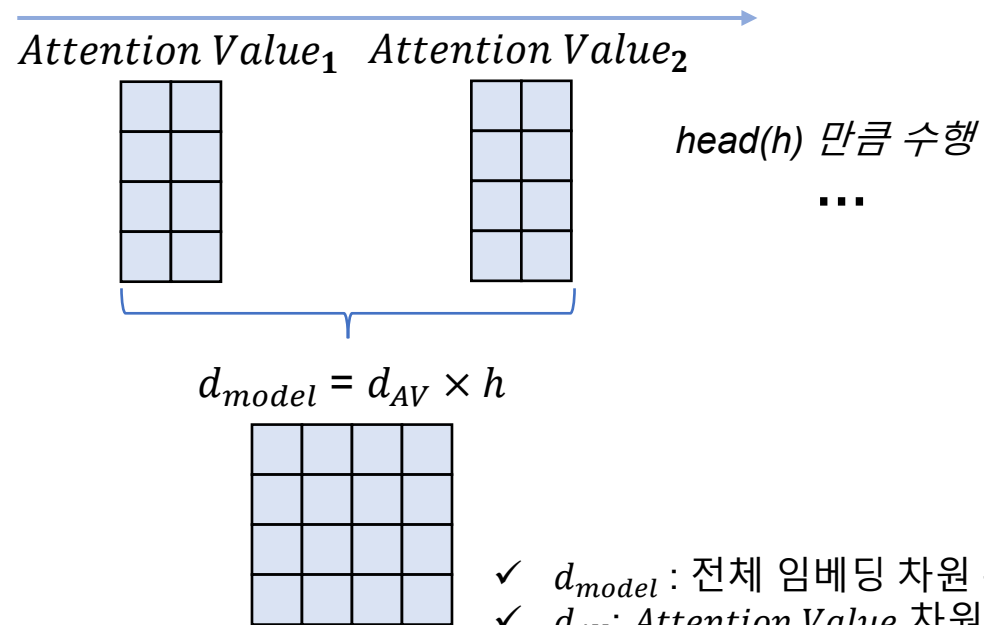
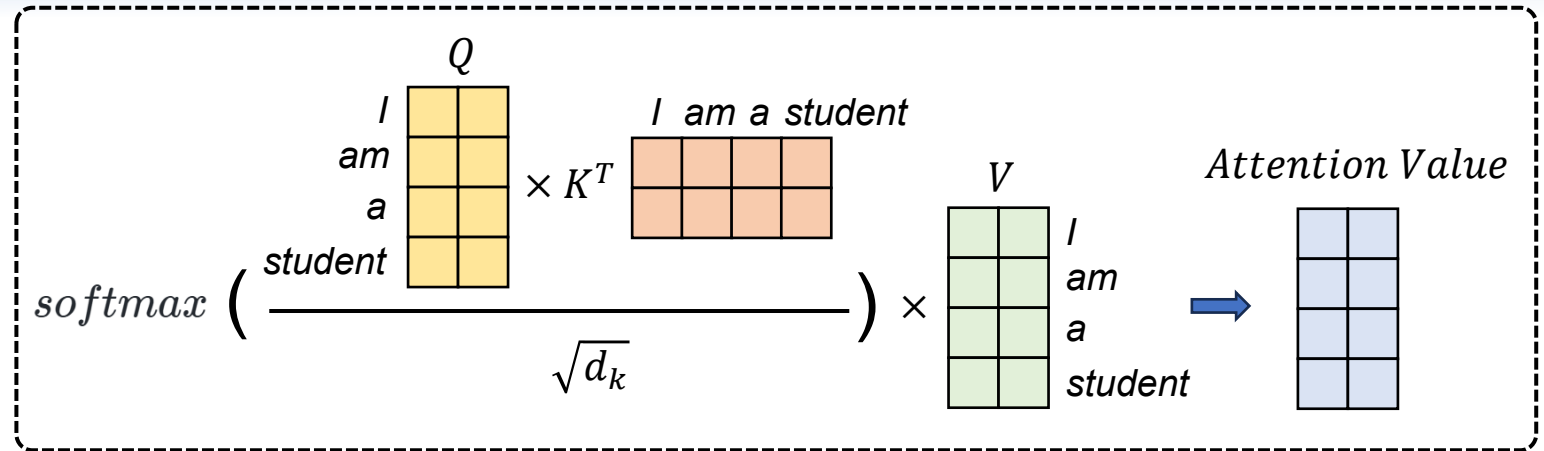
Encoders

- Multi-head Self-Attention
- Scaled Dot-Product Attention

Ex.) I am a student



Multi-head Self-Attention



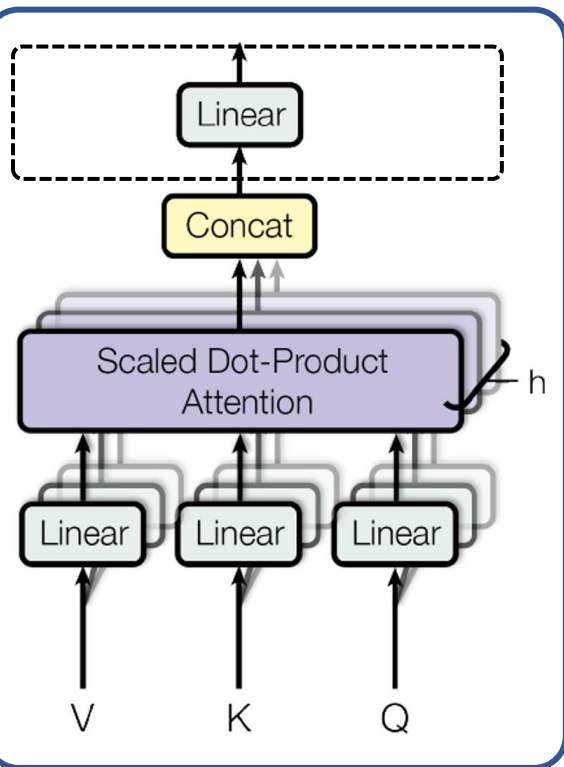
- ✓ d_{model} : 전체 임베딩 차원 수
- ✓ d_{AV} : Attention Value 차원 수
- ✓ $head(h)$: 2로 가정

Transformer

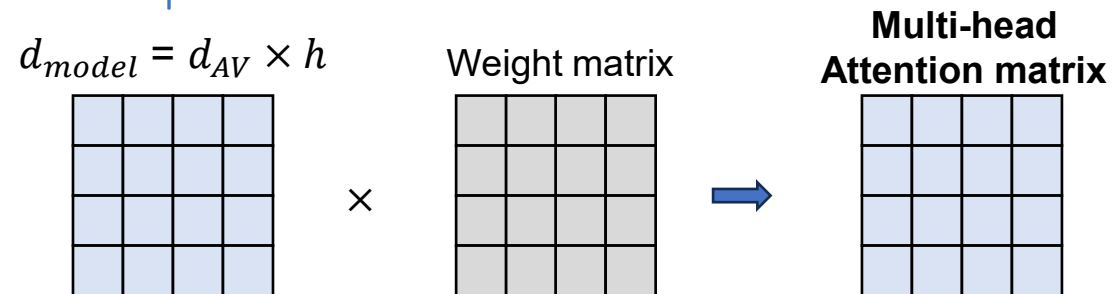
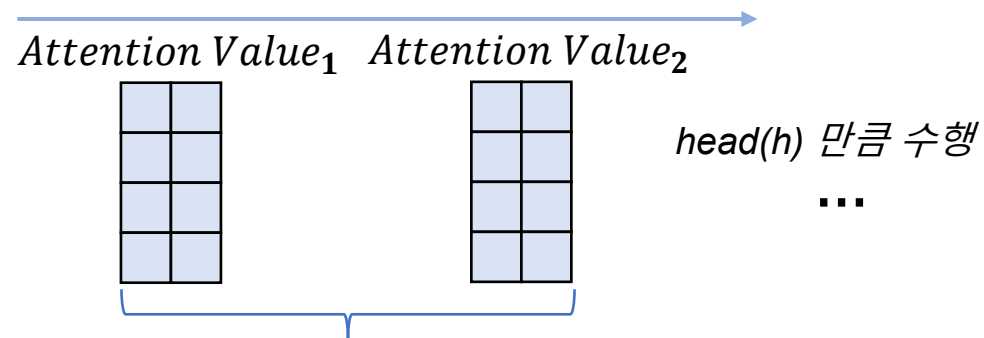
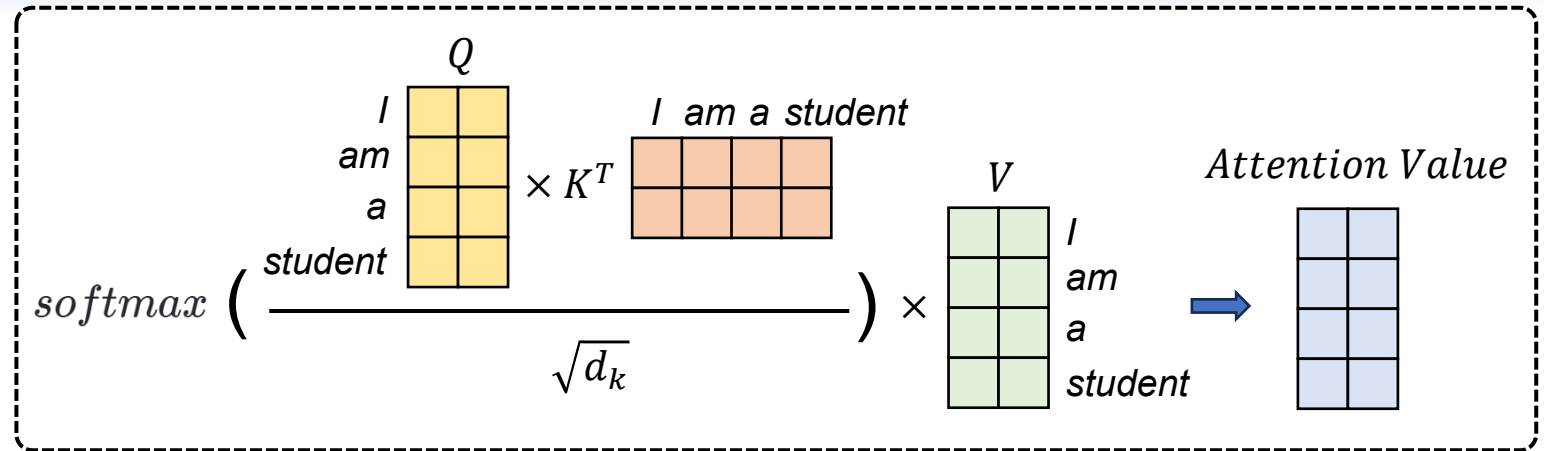
Encoders

- Multi-head Self-Attention
- Scaled Dot-Product Attention

Ex.) I am a student

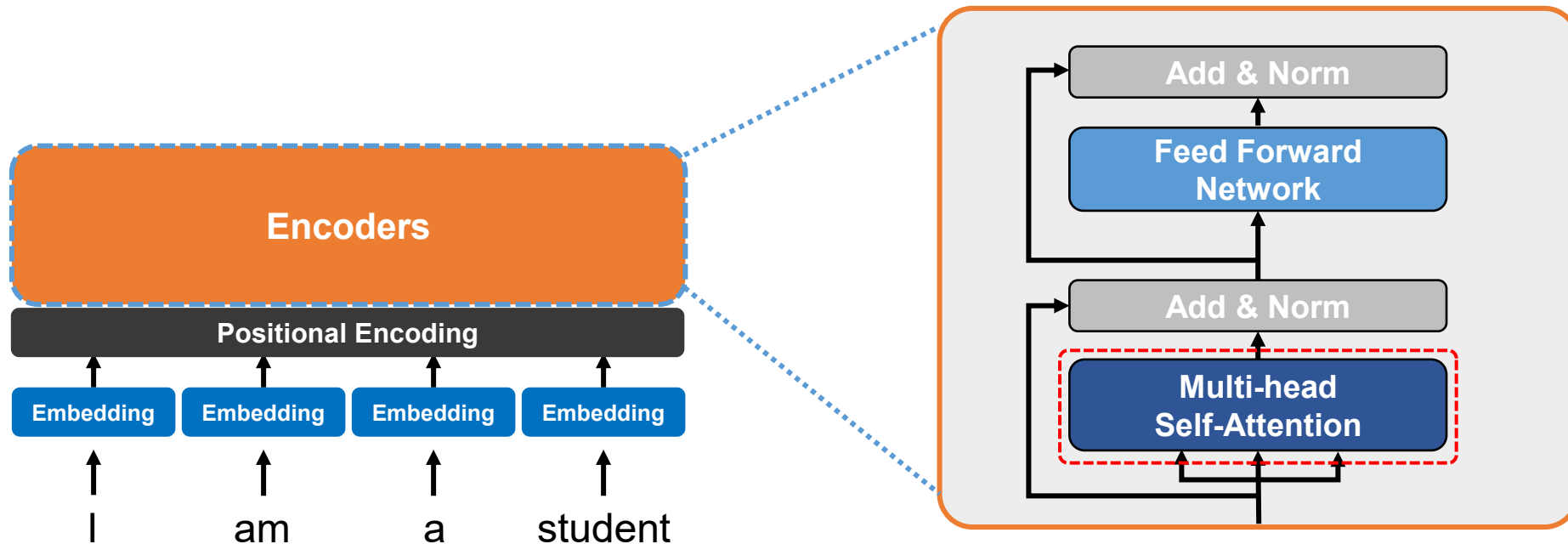


Multi-head Self-Attention



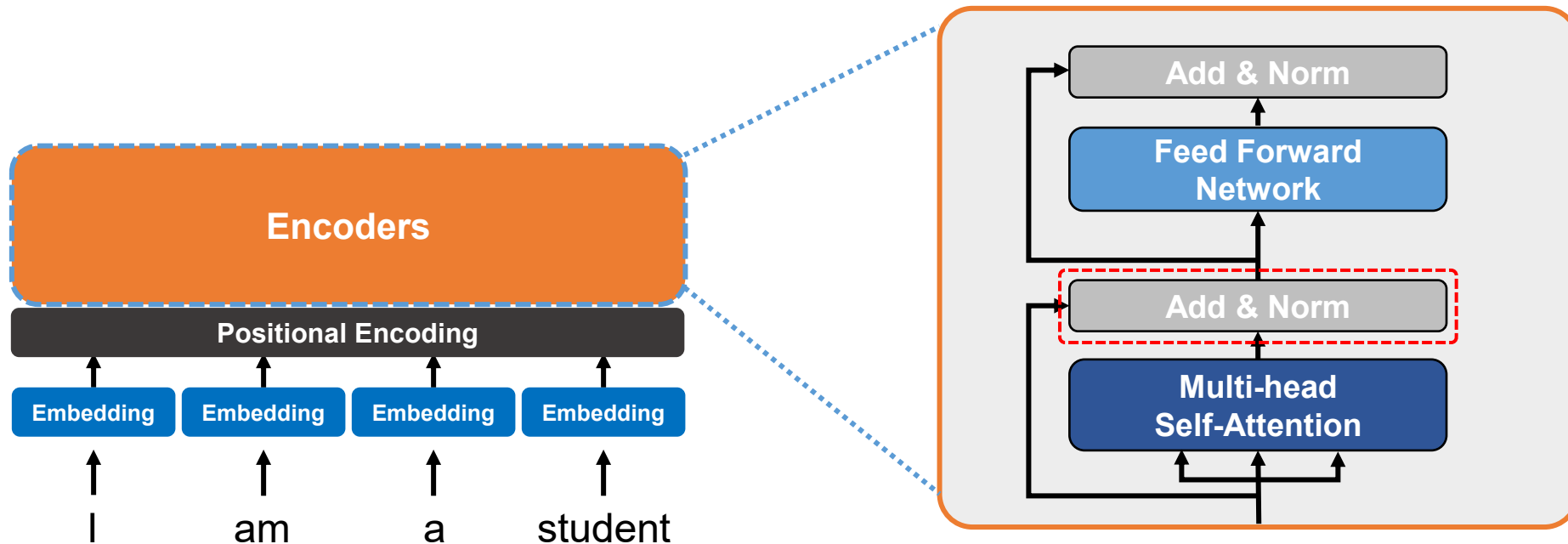
Transformer

- Encoders
 - Multi-head Self-Attention



Transformer

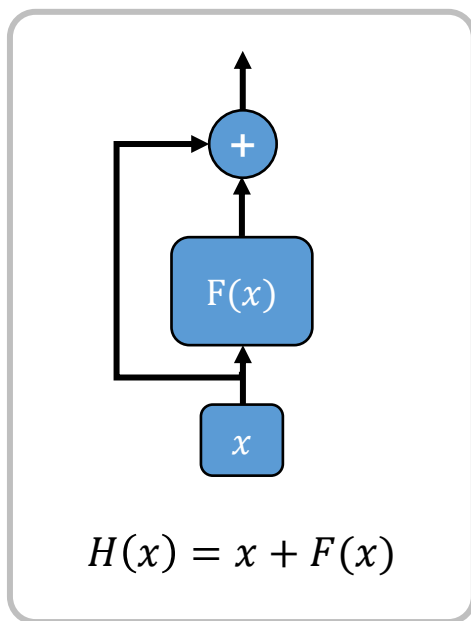
- Encoders
 - Residual Connection & Layer Normalization



Transformer

Encoders

- Residual Connection & Layer Normalization

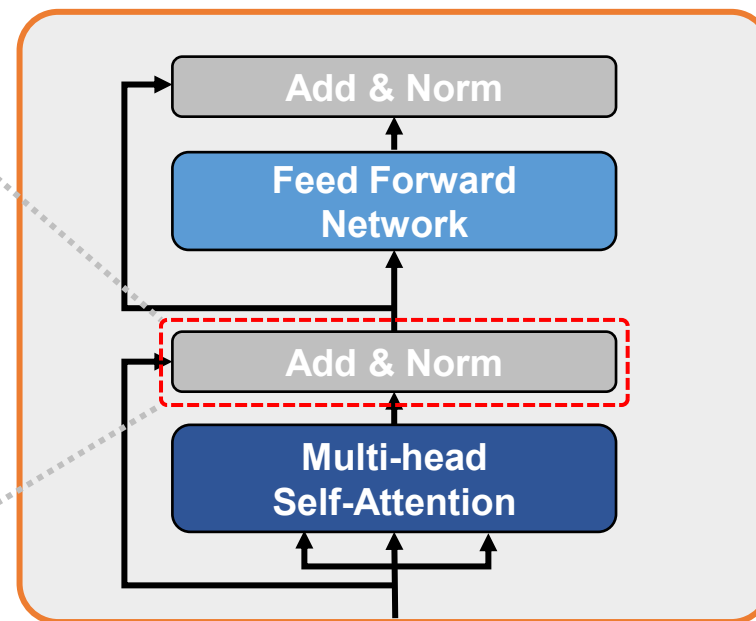


Residual Connection

$$\hat{x}_{i,k} = \frac{x_{i,k} - \mu_i}{\sqrt{\sigma_i^2 + \epsilon}}$$

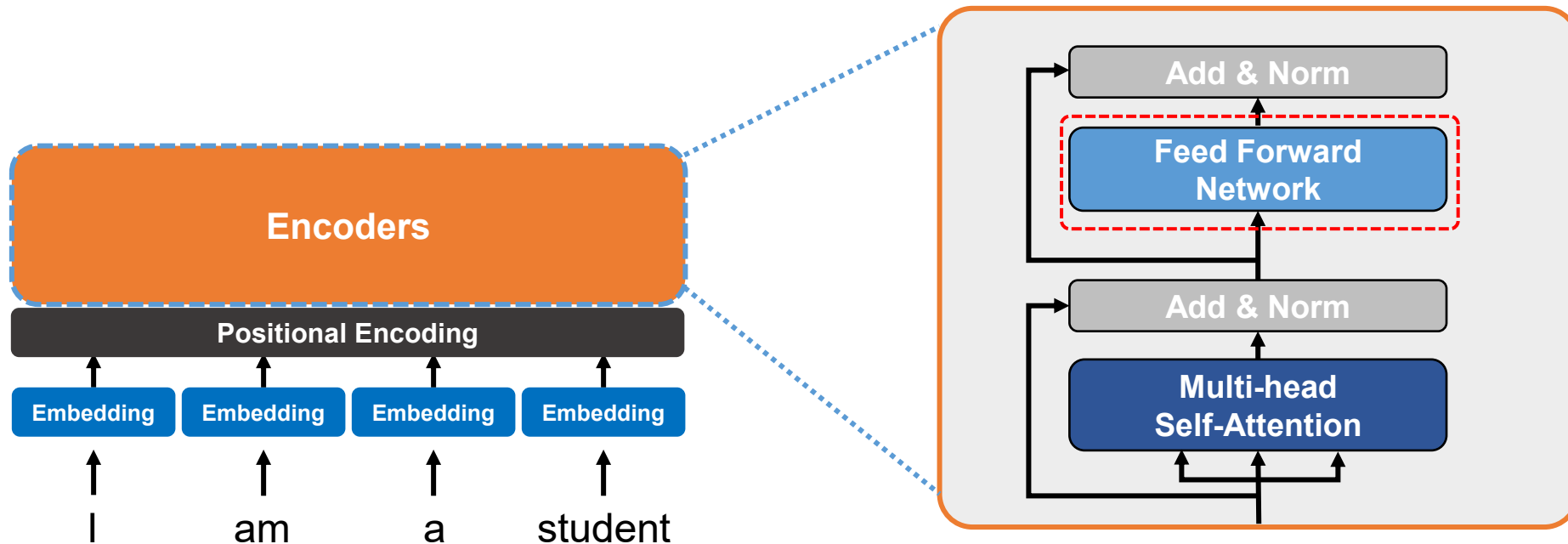
- ✓ x_i : 입력벡터
- ✓ k : 벡터 x_i 의 각 차원
- ✓ μ_i : 평균
- ✓ σ_i^2 : 분산
- ✓ ϵ : 분모가 0이 되는 것을 방지

Layer Normalization



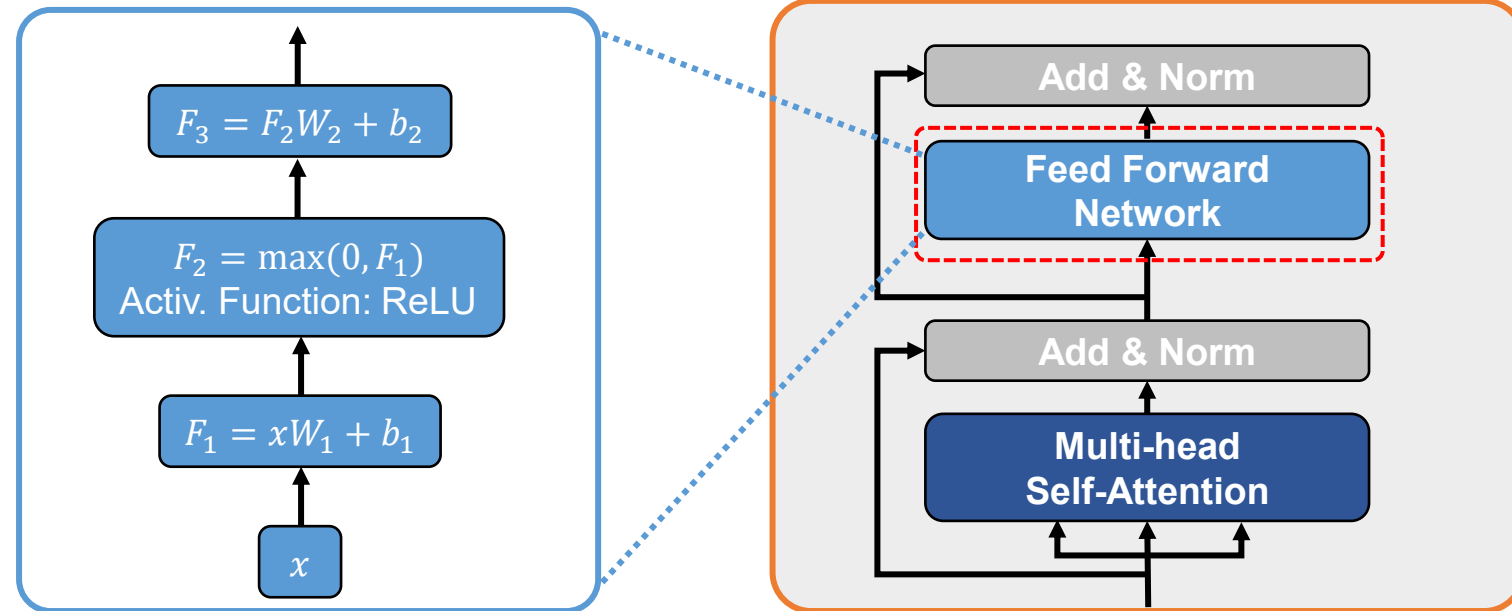
Transformer

- Encoders
 - Feed Forward Network



Transformer

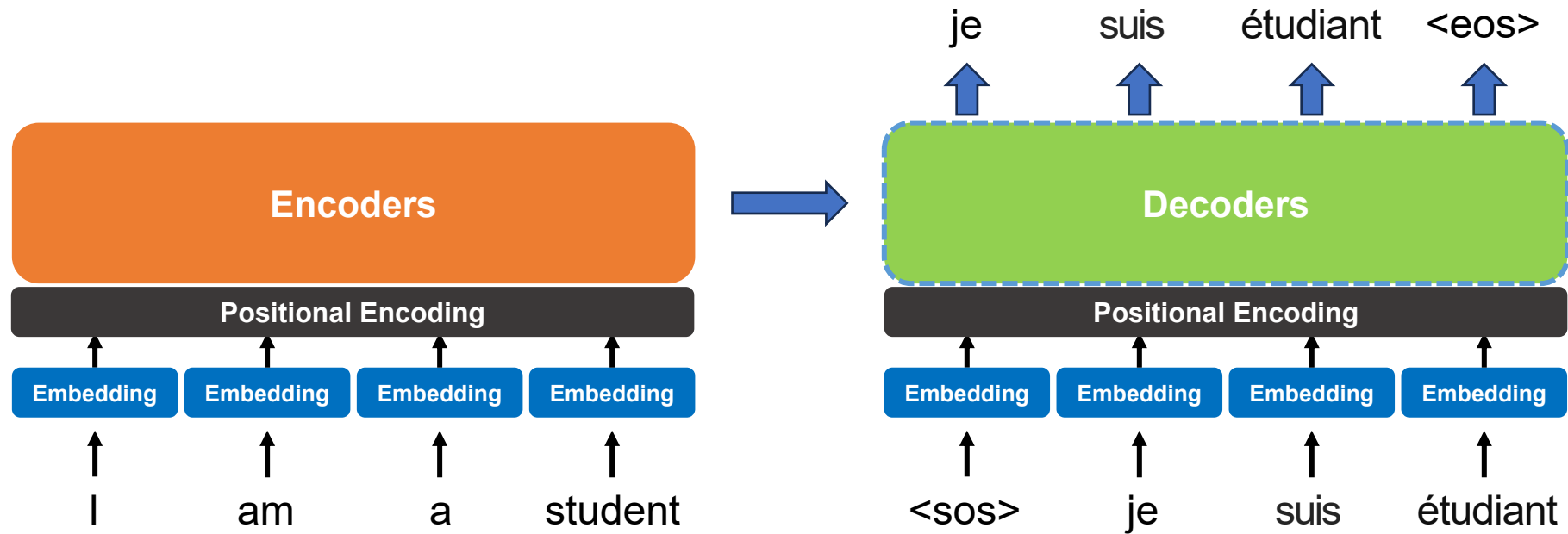
- Encoders
 - Feed Forward Network



$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

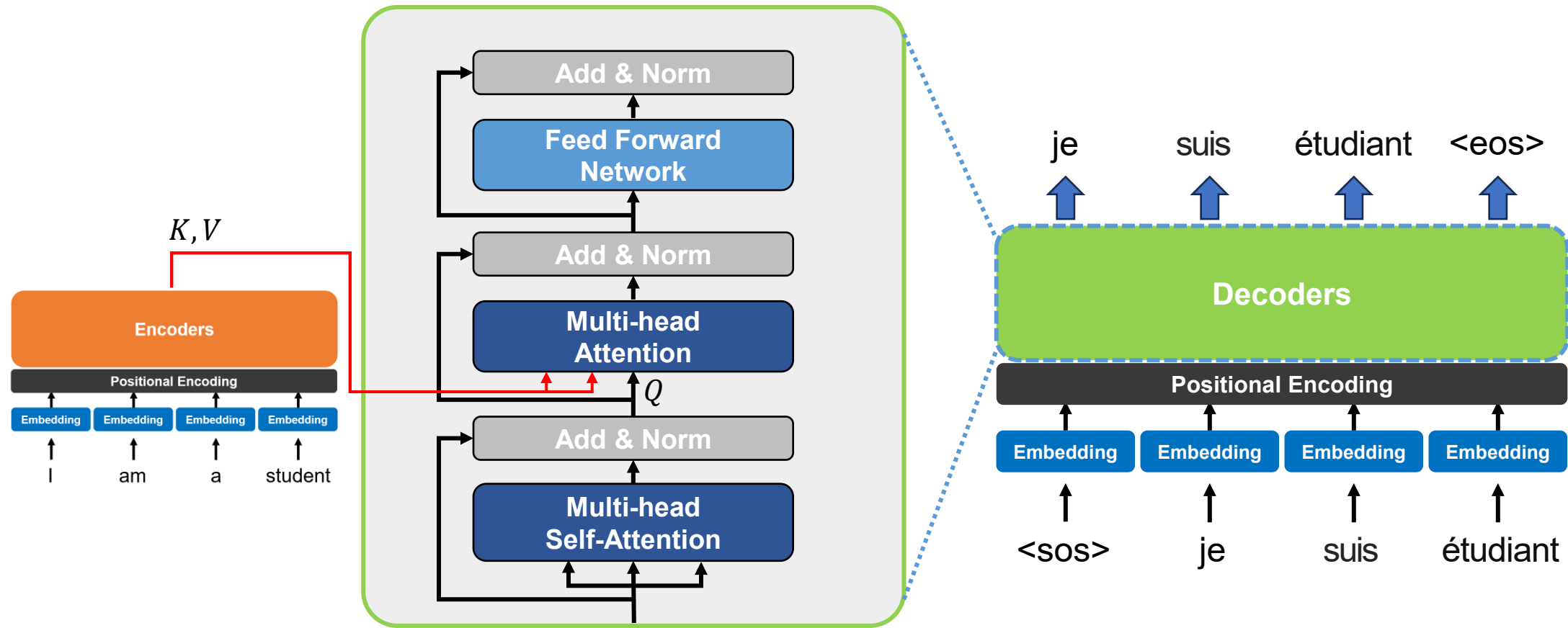
Transformer

- Decoders



Transformer

- Decoders

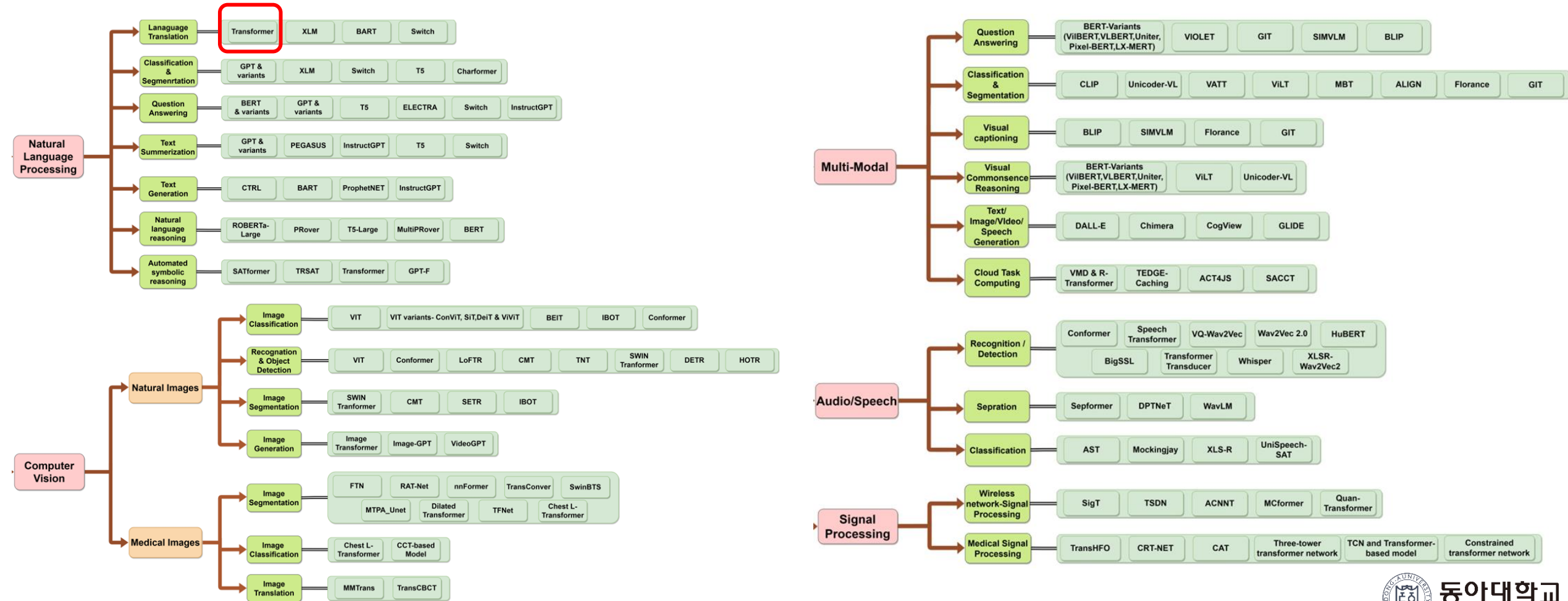


Transformer



Transformer

- Transformer 등장 이후 많은 분야에서 연구



Transformer

- Transformer 등장 이후 많은 분야에서 연구

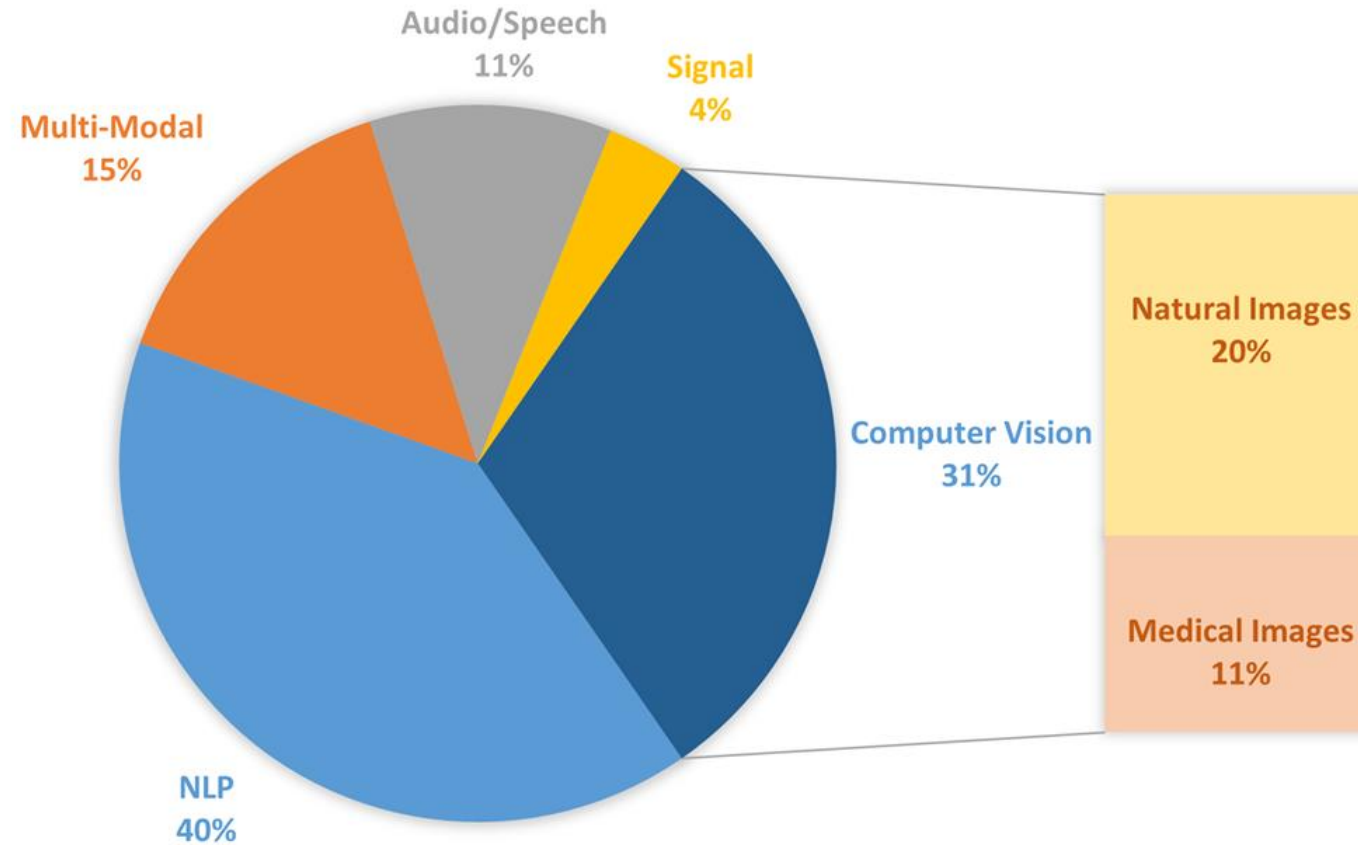


Figure 4: Proportion of transformer application in Top-5 fields

Questions & Answers

Dongsan Jun (dsjun@dau.ac.kr)

Image Signal Processing Laboratory (www.donga-ispl.kr)

Division of Computer·AI Engineering

Dong-A University, Busan, Rep. of Korea